



Degree Project in Embedded System

Second cycle, 30 credits

Evaluating a RISC-V processor running Benchmarks using the QEMU Virtual Platform tool.

GENGWU DU

Evaluating a RISC-V processor running Benchmarks using the QEMU Virtual Platform tool.

GENGWU DU

Master's Programme, Embedded Systems, 120 credits

Date: November 1, 2022

Supervisor: Mark Smith

Examiner: Johnny Öberg

School of Electrical Engineering and Computer Science

Swedish title: Evaluating a RISC-V processor running Benchmarks using the QEMU Virtual Platform tool.

Abstract

In recent years, developers have wanted to design more complex and advanced embedded processors. The feasibility of developed processors must be verified before the actual application. However, the process of verification always needs high costs and time. Quick Emulator (QEMU), a virtual platform emulator, can help in this situation. It can emulate different processors and hardware environments and build a unique platform according to the designers wishes. Many people have used QEMU to emulate advanced Reduced Instruction Set Computing (RISC) processors (like ARM) or the X86 architectures. Still, there is little research on RISC-V processors. Therefore, studying the QEMU to emulate the RISC-V processor is important.

This thesis aims to evaluate the performance of the RISC-V processor using QEMU. Ten different benchmarks are designed, and their results are compared to reflect the performance of the RISC-V as well as the simulator. These results provide a reference when these benchmarks are run on the RISC-V processor that is on the actual hardware development board.

Keywords

QEMU, RISC-V, Benchmark, Virtual Platform, GDB, Bare metal

Sammanfattning

De senaste åren har utvecklarna alltid velat utveckla mer komplexa och avancerade funktioner på inbyggda utvecklingskort. Men de nya funktionerna måste verifieras innan man gör kretskortet. Å andra sidan så kostar verifieringsprocessen mycket tid och pengar. Quick Emulator (QEMU), en virtuell plattformsemulator, kan hjälpa till för att lösa detta problem. Den kan emulera olika processorer och hårdvarumiljöer och bygga en unik plattform allt enligt designernas önskemål. Många människor har använt QEMU för att emulera avancerade Reduced Instruction Set Computing (RISC)-processorer (som ARM), eller X86-arkitekturerna, men det finns mycket lite forskning om RISC-V processorer. Därför är det viktigt att studera QEMU för att emulera RISC-V-processorn.

Denna avhandling syftar till att utvärdera prestandan för RISC-V processorer genom att använder QEMU. Tio olika benchmarks konstrueras för att användas för att spegla prestandan hos processorn såväl som simulatorn. Dessa resultat kan sedan användas som referens när benchmarken körs på de RISC-V-processorer som finns på det aktuella hårdvaruutvecklingskortet.

Nyckelord

QEMU, RISC-V, Benchmark, Virtual Platform, GDB, Bare metal

Acknowledgments

I want to thank my examiner Johnny Öberg and my supervisor Mark Smith for their support and suggestions during my thesis project. Without you, I cannot keep the process of my thesis. I do appreciate my family. They gave me a lot of support in continuing my studies in a foreign country. I want to express the depth of my gratitude to my friends Ziyang He, Annan Liu, Mingda Sun, Yuhang Tian and Tianhao He. They give me many academic supports and helped me revise my thesis. Please accept my heartfelt appreciation.

Stockholm, November 2022

Gengwu Du

Contents

1	Introduction	1
1.1	Motivation and Importance	1
1.2	Problem statement	2
1.3	Research Questions and Objectives	2
1.4	Structure of the thesis	3
2	Background	5
2.1	QEMU	5
2.1.1	The internals of QEMU	6
2.1.2	QEMU mode	7
2.2	Benchmark	8
3	Installation of QEMU based on RISC-V	10
3.1	The riscv-gnu-toolchain	10
3.1.1	Pre-requisites	10
3.1.2	Getting the source	11
3.1.3	Setting config file	11
3.1.4	Build the compiler	12
3.1.5	Adding as a global variable	14
3.2	Installation of QEMU	14
3.3	The installation of Linux	15
3.4	The installation about busybox	16
3.5	Making root filesystem	16
4	Running QEMU and compiler	19
4.1	Running compiler	19
4.2	Running QEMU	20

5	Benchmark	23
5.1	Fibonacci sequence	23
5.2	Dhrystone	24
5.3	Linpack	26
5.4	Speed	26
5.5	Matrix multiplication	28
5.6	Signal filter	28
5.7	Neural Networks	30
5.8	Imageproc	31
5.9	Fast Fourier transform	34
5.10	Compression	35
6	“Virt” platform analysis	38
6.1	Memory space distribution	39
6.2	Starting process of “virt” platform	39
6.3	The loading about the firmware and kernel	40
7	The bare-metal test on QEMU	41
8	Results and discussions	46
8.1	Test environment	46
8.2	Fibonacci sequence	47
8.3	Dhrystone	48
8.4	Linpack	49
8.5	Speed	50
8.6	Matrix multiplication	51
8.7	Signal filter	52
8.8	Neural Networks	53
8.9	Imageproc	54
8.10	FFT	54
8.11	Compression	56
8.12	Conclusion	57
9	Conclusion and Future Works	58
	References	59

List of Figures

2.1	The QEMU architecture layer overview	5
2.2	The internal QEMU architecture[10]	6
2.3	The overview of the TCG[10]	7
3.1	Installation process of Compiler	11
3.2	The list of compiler tools	13
3.3	Result of installation of the Linux kernel	15
3.4	The interface about Busybox Configuration	17
3.5	The structure of root file system directory	18
4.1	The initial interface of QEMU	22
4.2	The console of Linux kernel on QEMU	22
5.1	The logical block diagram about Dhrystone	25
5.2	The logical block diagram about speed	27
5.3	The direct-form structure of the FIR filter[27]	29
5.4	The structure of the Neural network	31
5.5	The input picture of grayscale conversion	32
5.6	The output picture of grayscale conversion	32
5.7	The input picture of image sharpening	34
5.8	The output picture of image sharpening	34
5.9	The structure of Huffman tree	37
5.10	The test figure "Lena"	37
7.1	The interface of GDB	45
7.2	The interface of QEMU	45
8.1	The result of Neural Networks	53
8.2	The result of Image processing	55

Chapter 1

Introduction

This chapter introduces the research topic that the thesis focuses on and some questions that should be solved. The following states this thesis's motivation, problem description, research objectives, research method and thesis outline in detail.

1.1 Motivation and Importance

Embedded processors are used in many domains such as vehicles, communication, healthcare, and more. In recent years, more and more complex and advanced embedded processors have been designed. On the other hand, the feasibility of developed processors must be verified before the actual application is built. So, the process of verification always needs to make a new prototype and build a suitable hardware environment. This causes the cost to increase. Therefore, it is necessary to use more advanced techniques to reduce the cost of building solid hardware for verification. With the adoption of hardware virtualisation, people can set the memory size flexibly and add serial ports or devices needed to realize a system that fulfils their goals. Hence, people can set the hardware environment flexibly, reduce the cost of purchasing different hardware development boards, and improve work efficiency [1]. In this project, the tool for hardware virtualisation is QEMU, an open-sourced platform[2] that can be customized to emulate different processors and hardware environments[3].

For the different processors supported by the QEMU, processors based on ARM and x86 architectures are widely used. Many people have researched these two kinds of processors on the QEMU. But for the processor based on the RISC-V architecture, people's research on the emulation of this processor

on QEMU is not exhaustive. Unlike to the processors based on the ARM or x86 architectures, the instruction set architecture (ISA) of the RISC-V is open-source. The developers do not need to pay for the RISC-V ISA, so any developer can develop their own version of it[4]. Therefore, more developers are willing to spend time researching it. Moreover, as more and more people start exploring and using it, it is bound to become more popular and more successful and become a trend of future development. So, it is fruitful to use RISC-V as a research object. Therefore, this thesis wants to use the QEMU to emulate a computer platform which is built with a RISC-V processor and test the performance of the RISC-V processors.

1.2 Problem statement

In order to test the performance of the RISC-V processor on QEMU, the initial task is to configure the QEMU that provides a RISC-V processor. It is a significant step. Only after this step is completed, the next test process can continue.

The other question is that the benchmarks are written in C language, and it is impossible to run them directly for the QEMU. To solve this problem, a suitable compiler is necessary. This compiler first needs to convert those benchmarks into executable files that can be run on the QEMU and this compiler needs to support different RISC-V instruction sets.

Testing the performance of the processor on QEMU is a critical task in this thesis. Performance tests include whether the processor supports a variety of data types and operation types based on different RISC-V instruction sets. To finish a comprehensive task, the designed Benchmark needs to contain various data types and algorithms. Furthermore, these benchmarks are compiled based on different RISC-V instruction sets. All of the compiled benchmarks will be run on the QEMU and the results compared with running the actual processor.

1.3 Research Questions and Objectives

For this thesis, the questions are explored in the following chapters:

- How to install the QEMU that focuses on RISC-V processors?
- What kind of benchmark will be designed?
- RISC-V instruction sets include many types. What kinds of RISC-V instruction set combinations should be chosen by the compiler so

that the generated file can be used to test the RISC-V processor more comprehensive.

For this thesis, these problems will be solved by the following objectives:

- A tutorial about the installation of the QEMU for the RISC-V on the official website of QEMU [5], which is helpful about how to install the QEMU for RISC-V processors
- For types of benchmark, this paper plans to design ten different benchmarks to analyze the performance of QEMU operations. The specific information will be mentioned in the following chapters. The main strategy is to focus on a certain type of benchmark with integers operations, some benchmarks focus on operations on floating-point numbers, and some have both.
- For the instruction set of RISC-V in this thesis, the main types mentioned that is Integer(I), Multiplication and Division(M), and Single-Precision Floating-Point(F) and Double-Precision Floating-Point(D). The main method to ensure the test set is complete is controlling variables. For each benchmark, it is divided into:
 - Integer(I)
 - Integer(I) and Multiplication and Division(M)
 - Integer(I), Multiplication and Division(M) and Single-Precision Floating-Point(F)
 - Integer(I), Multiplication and Division(M), Single-Precision Floating-Point(F) and Double-Precision Floating-Point(D)

Furthermore, the compiler executes based on the 32-bit and 64-bit RISC-V instruction sets, and then the generated executable file kicks in and runs in the QEMU emulator for testing.

1.4 Structure of the thesis

Including the introduction, this paper contains a total of 8 chapters. Chapter 2 mainly focuses on the related introductions of QEMU, the typical benchmarks for the embedded platforms, and some technical details. Chapter 3 mainly introduces the installation of the emulator and the compiler. Some of the more critical commands are introduced and explained. Chapter 4 explores how to

run QEMU and the compiler. It mentions many script files and arguments required during the installation and operation of QEMU. The design principles of ten benchmarks, including the main data and operation types, are explained in Chapter 5. Chapter 6 contains information on how the “virt” platform works for the QEMU emulator. The “virt” platform is an essential platform for the QEMU emulator. Chapter 7 records using QEMU to run benchmarks in bare metal conditions. Chapter 8 mainly states the results of running the benchmark in the QEMU emulator and analyses these results. Chapter 9 summarises the project and what needs to be done in the future.

Chapter 2

Background

2.1 QEMU

According to Fabrice Bellard, QEMU’s full name is Quick Emulator[6], an open-source, quick, cross-platform machine emulator[7]. In figure 2.1, an overview of the QEMU architecture is shown. The QEMU is a virtual machine monitor for the host. It can emulate a Central Processing Unit(CPU) architecture that does not need to match the architecture of the host processor by using the dynamically binary translation[8][9]. A new operating system can be installed on the emulated processor, which cannot be affected by the host operating system.

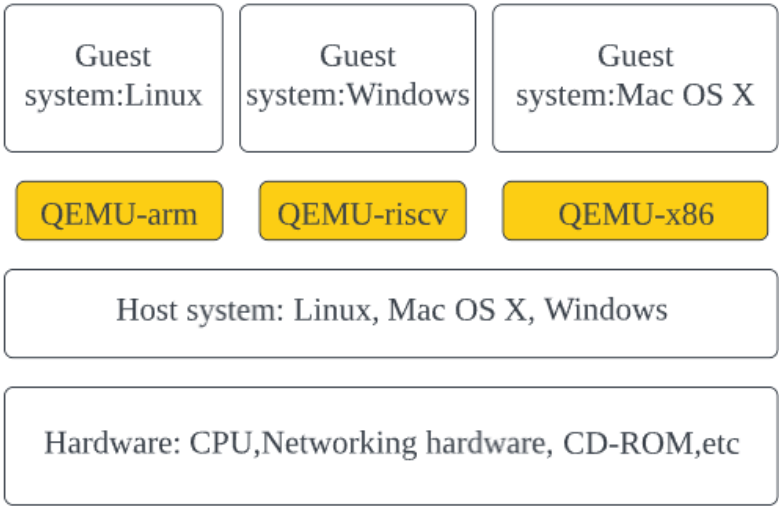


Figure 2.1: The QEMU architecture layer overview

Furthermore, QEMU also simulates a series of hardware models, making the guest operation system think it is interacting with real hardware. Still, this hardware is just simulated by QEMU. The basic working principle is that when the guest operation system interacts with these virtual hardware models, QEMU will translate these instructions into the commands pool corresponding to the host operation system so that the actual hardware can receive the information in the instructions and respond accordingly. For example, the guest system is Windows and this operation system wants to access the Compact Disc Read-Only Memory(CD-ROM). Then the QEMU will translate this instruction to the new instruction that the host system can understand. After that, the host system will access the CD-ROM and the CD-ROM receives this command and responds accordingly.

2.1.1 The internals of QEMU

According to Hannes[10], the internal structure of the QEMU is shown in Figure 2.2.

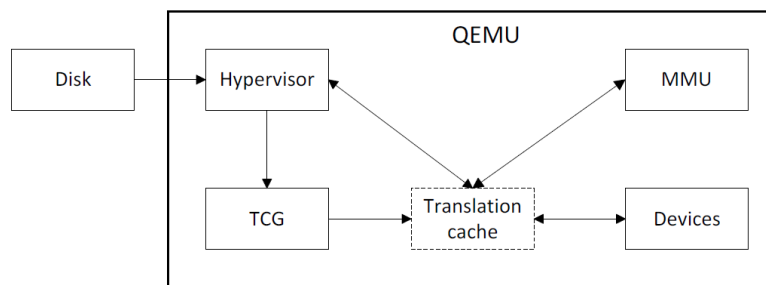


Figure 2.2: The internal QEMU architecture[10]

According to Figure 2.2, the specific workflow of the QEMU has five steps[10]:

- Hypervisor(virtual machine monitor) gets the binary machine code which is from the disk image.
- Hypervisor asks the TCG(Tiny Code Generator)to convert these binary machine commands into an executable format for the virtual machine. These translated machine codes are stored in the translation cache.
- By the translation cache, the devices which are virtual or real can be connected with the hypervisor.

- By the translation cache, the hypervisor can start the software MMU(Memory Management Unit).
- After completing the above process, the hypervisor will emulate the Operating System(OS) stored on the disk image.

In this process, the TCG and MMU are the critical parts considered keys to realise the virtualisation of the processor and memory, respectively. Due to the specific processor, the instruction set only can run on a fixed type of processor. For instance, it is impossible to run the RISC-V instruction set in the ARM processor. This model aims to translate the non-native source machine codes to the native source machine codes. The overview of the tiny code generator is shown as follows.

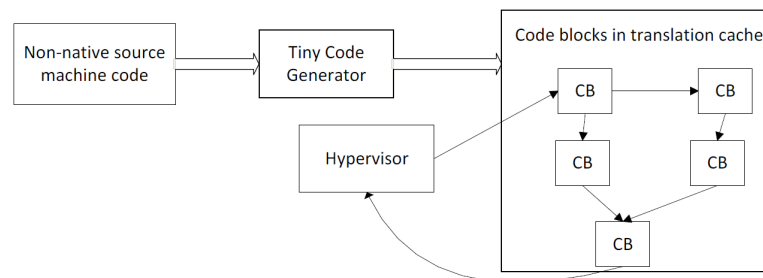


Figure 2.3: The overview of the TCG[10]

In the second step of the specific workflow of the QEMU, the non-native source machine codes are bonded into blocks by the TCG and these code blocks are connected with the instruction Jump as shown in the figure 2.3[10]. When the hypervisor wants to run some codes, the instruction jump will help the hypervisor to jump into the code blocks. And if the new code block needs to be translated, the hypervisor will tell the TCG to translate more codes to generate a new code block. Similar to traditional hardware MMU, it is designed to convert virtual addresses to physical addresses. For the software MMU, it can increase the translation speed by addressing the translation cache which includes the guest addresses, host addresses and offset values[11].

2.1.2 QEMU mode

There are two different emulation modes: user-mode emulation and system emulation[8].

- User mode

In this mode, the single process which should run in the guest processor can run in the host processor[11]. The emulation mode is only a part of the system mode emulation because it does not emulate the MMU but uses the host operating system to deal with the single process[3]. Therefore, the efficiency of this mode is higher than the system mode. Nevertheless, only Linux and Berkeley Software Distribution(BSD) support this emulation mode.

- System mode

This mode is the authentic QEMU emulation during which the whole computer system is emulated. It includes the CPU and peripherals. As the definition of the QEMU, dynamic binary translation can emulate the CPU[9]. And by some specific commands, a developer can specify some board models to emulate[12]. Nearly all Instruction set architectures and operating systems are compatible.[8]

2.2 Benchmark

The main aim of this thesis is to test the performance of the QEMU. Therefore, using some benchmarks is necessary. According to David Patterson [13], the development of benchmarks has a long history, and there are a total of seven criteria for the establishment of embedded benchmarks:

- The embedded benchmark needs to be free so that it can be widely used.
- The embedded benchmark should run and port easily.
- The embedded benchmark should use a group of programs.
- The embedded benchmark must be supported by an organization that can maintain its relevance over time.
- The embedded benchmark should report a performance score.
- The embedded benchmark should use geometric mean and standard deviation for summarizing.
- The embedded should be suitable for the field of academia and industry.

For the benchmark for this thesis, a total of ten programs exist in the benchmark, each of which is written in the C or C++ Programming Language. By GNU Compiler Collection(GCC), these programs could be compiled for different computer architectures. Hence, it is easy to run and deploy. Because the QEMU is run in a Linux environment, the Linux command “time” can show how long this program runs. In addition, experimental replication is required to obtain a convincing performance score.

Furthermore, to test the RISC-V processor more comprehensive, various programs are designed. These programs aim at different data types or different operating types. The details of the benchmark are shown in Chapter 4.

Chapter 3

Installation of QEMU based on RISC-V

In this chapter, according to the started guide provided by the RISC-V official website[14], the installation of the QEMU is introduced. To ensure that QEMU can run perfectly, five components must be installed and configured correctly: the cross compiler, QEMU, Linux image file, Busybox and root file system. Some specific commands are required in order to finish the installation of the QEMU. The details about these components are shown as follows.

3.1 The riscv-gnu-toolchain

The project runs on Ubuntu 18.04, but the QEMU in this paper is based on the RISC-V architecture. Hence, it is necessary to use a cross-compiler to generate the code that can run on the QEMU. The riscv-gnu-toolchain [15] is an open-source project that introduces a cross-compiler focusing on the RISC-V ISA. It can compile the C or C++ programs and generate executable files whose format is Executable and Linking Format(ELF) that can run on the RISC-V processor. And there are two kinds of compiler modes that can be used based on the two different C function libraries. The flow chart of the installation process is shown in figure 3.1. It includes five main steps during the installation of the cross-compiler.

3.1.1 Pre-requisites

To make sure the compiler can run perfectly, several standard packages need to be installed to support running this compiler. In Ubuntu, the command needed

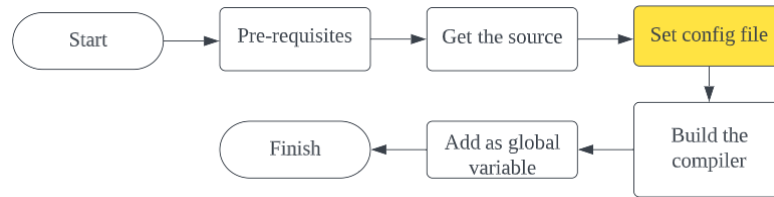


Figure 3.1: Installation process of Compiler

to input in the terminal is

```

sudo apt-get install autoconf automake \
autotools-dev curl python3 libmpc-dev libmpfr-dev \
libgmp-dev gawk build-essential \
bison flex texinfo gperf libtool \
patchutils bc zlib1g-dev libexpat-dev

```

And for version 18.04 of Ubuntu, another package called *ninja-build* is needed. But if Ubuntu's version is higher than version 18.04, the package *ninja-build* has already been installed.

3.1.2 Getting the source

In Linux operating system, this suitable compiler can be downloaded by command:

```
git clone https://github.com/riscv/riscv-gnu-toolchain
```

This downloaded file folder named as *riscv-gnu-toolchain* is a huge file folder which occupies about 6.65GB. It consists of all of the things about the compiler. Afterwards commands about compiler installation must be executed in this folder.

3.1.3 Setting config file

This step is the most crucial part. It can decide what kind of instruction set the compiler will support and the location of the compiler download. This step can be finished by commands:

```

./configure --prefix=$HOME/Desktop/riscv \
--with-arch=rv32i\
--with-multilib-generator="rv32i-ilp32--;\
rv32im-ilp32--;rv32imfd-ilp32d--;

```

Where the parameter “*-prefix=\$PATH*” is used to determine the address of the generated compiler. The parameter “*-with-arch*” is used to determine whether the compiler focuses on 64-bit or 32-bit. And the parameter “*-with-multilib-generator=*” can be used to determine what kind of RISC-V ISA is needed. The types of instructions involved in this thesis are i(nteger), (m)ultiplication and division, (f)loat and (d)ouble. This compiler also supports another instruction sets that can be found in the official document [15]. If the compiler is required to compile files for a 64-bit RISC-V processor, the user only needs to change the numerical variable from 32 to 64 in this command.

3.1.4 Build the compiler

After finishing the above steps about compiler installation, the last step is to run the Makefile. There are two modes of the compiler in this open-sourced project. They have different prefixes, which are “*riscv64-unknown-elf-*” and “*riscv64-linux-gnu-*”. If the compiler only for RISC-V32 is selected after setting the configuration, the generated compilers are “*riscv32-unknown-elf-*” and “*riscv32-linux-gnu-*” prefixed versions, respectively. These two modes can be caused by inputting the command “*make*” and “*make linux*” in the terminal respectively. About the two modes of the compiler, they have different C runtime libraries. The detail will be shown as follow:

- The compiler whose version is prefixed with *riscv64-unknown-elf-* represents this compiler as a non-Linux version. In addition, this kind of compiler can also run on the Linux operation system with the help of NewLib which is a C runtime library[16]. Newlib is a C library intended for using in embedded systems[17]. It is a collection of several libraries, including C Standard, a math library and misc. It cannot replace the library for operating systems.
- The compiler whose version is prefixed with *riscv64-linux-gnu-* represents this compiler as the Linux version. Similar to the first type (in .elf format), the .gnu formatted one uses Glibc as the C runtime library. Glibc is a complete C runtime library. It nearly includes all functions from another version of the C runtime library. And this library even can replace the native system’s library completely.[18]
- Both compilers have some flaws. For the first mode, the library does not support the function “*malloc()*” for the C programs. Hence, some benchmarks written must use the second mode of the compiler. But the

second version makes the executed file bigger than the first version for the second mode. It could affect the performance of the QEMU.

- In addition, the instruction set combination designed in this project cannot be realised in this mode. When the combination of integer(I) and Multiplication and Division(M) is set in the steps of configuration, the Makefile will run with some errors by the command “*make linux*”. It must have atomic(A) instructions.

The strategy used in this thesis adopts the first mode as the main method to compile the C program. When the C program uses limited functions that cannot run with the first mode, the second mode generates the executed file. After finishing this step, the list of the tools can be shown in the file “*bin*” of the “*riscv*”. The specific content is shown in the figure 3.2

```
dgw@dgw-VirtualBox:~/Desktop/riscv/bin$ ls
riscv64-unknown-elf-addr2line      riscv64-unknown-linux-gnu-ar
riscv64-unknown-elf-ar             riscv64-unknown-linux-gnu-as
riscv64-unknown-elf-as             riscv64-unknown-linux-gnu-c++
riscv64-unknown-elf-c++           riscv64-unknown-linux-gnu-c++filt
riscv64-unknown-elf-c++filt       riscv64-unknown-linux-gnu-cpp
riscv64-unknown-elf-cpp           riscv64-unknown-linux-gnu-elfedit
riscv64-unknown-elf-elfedit       riscv64-unknown-linux-gnu-g++
riscv64-unknown-elf-g++           riscv64-unknown-linux-gnu-gcc
riscv64-unknown-elf-gcc           riscv64-unknown-linux-gnu-gcc-11.1.0
riscv64-unknown-elf-gcc-11.1.0   riscv64-unknown-linux-gnu-gcc-ar
riscv64-unknown-elf-gcc-ar        riscv64-unknown-linux-gnu-gcc-nm
riscv64-unknown-elf-gcc-nm        riscv64-unknown-linux-gnu-gcc-ranlib
riscv64-unknown-elf-gcc-ranlib    riscv64-unknown-linux-gnu-gcov
riscv64-unknown-elf-gcov          riscv64-unknown-linux-gnu-gcov-dump
riscv64-unknown-elf-gcov-dump     riscv64-unknown-linux-gnu-gcov-tool
riscv64-unknown-elf-gcov-tool     riscv64-unknown-linux-gnu-gdb
riscv64-unknown-elf-gdb           riscv64-unknown-linux-gnu-gdb-add-index
riscv64-unknown-elf-gdb-add-index riscv64-unknown-linux-gnu-gfortran
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-gprof
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-ld
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-ld.bfd
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-lto-dump
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-nm
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-objcopy
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-objdump
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-ranlib
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-readelf
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-run
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-size
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-strings
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-strip
riscv64-unknown-elf-gprof         riscv64-unknown-linux-gnu-addr2line
```

Figure 3.2: The list of compiler tools

According to figure 3.2, a list of tools corresponding to the two modes can be separated clearly. The main tools used are the tools “*gcc*”, “*g++*” and “*objdump*”. The tool about the GCC is used to compile the C program, g++ is used to compile the C++ program, and objdump can translate the executed binary file into readable format.

3.1.5 Adding as a global variable

To make these tools can run in every file, it is necessary to add this *”bin”* file as the global variable. For Ubuntu 18.04, it only needs to write the command: *“export PATH=\$PATH:\$HOME/Desktop/riscv/bin”* in the file named *.bashrc* and restart the system. Then these tools can be used in each file.

3.2 Installation of QEMU

The installation of QEMU is similar to that of the compiler. Firstly, some packages need to install. For Ubuntu, the installation command is:

```
sudo apt install autoconf automake autotools-dev\
curl libglib2.0-dev libmpc-dev libmpfr-dev \
libgmp-dev gawk build-essential bison flex \
texinfo gperf libtool patchutils bc zlib1g-dev \
libexpat-dev git pkg-config binutils-dev \
libboost-all-dev libssl-dev libpixmap-1-dev \
libpython-dev python-pip python-capstone \
virtualenv libncurses5-dev libncursesw5-dev
```

The commands to be executed afterwards are:

```
git clone https://github.com/qemu/qemu
git checkout v5.0.0
./configure --target-list=riscv64-softmmu, \
riscv64-linux-user
make
make install
```

Where

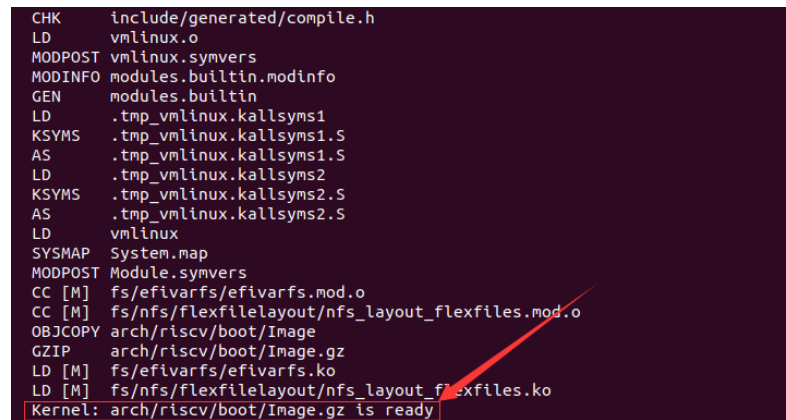
- The command *“git checkout”* can determine the version of QEMU.
- The configure command, *“--target-list”* can specify the architecture.
- The *“riscv64-linux-user”* is user mode, and *“riscv64-softmmu”* means complete system emulation.
- Finally, running *Makefile* by the command *“make”* and running the *“make install”* can place the built emulator and related files, such as the library, in the correct position.

3.3 The installation of Linux

This part aims to build a Linux kernel for the QEMU system mode. This part has three steps that need to be done. The commands to be executed afterwards are:

```
git clone https://github.com/torvalds/linux
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu-\
defconfig
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu-
```

where the first step is to download the source file. Secondly, using the command “*make defconfig*” creates a “*.config*” file with default options from the ARCH supplied defconfig. The file *defconfig* is used to provide the default options to generate the *.config* file in the same path. According to Colin Walls [19], Glibc could be used for the Linux kernel. So, the cross-compile is used with the *riscv64-unknown-linux-gnu-*. Finally, using the command “*make*” runs Makefile to complete the installation for the Linux kernel. And when the words shown in the figure 3.3 appear in the terminal, it proves that the Linux kernel has been generated.



```
CHK include/generated/compile.h
LD vmlinux.o
MODPOST vmlinux.symvers
MODINFO modules.builtin.modinfo
GEN modules.builtin
LD .tmp_vmlinux.kallsyms1
KSYMS .tmp_vmlinux.kallsyms1.S
AS .tmp_vmlinux.kallsyms1.S
LD .tmp_vmlinux.kallsyms2
KSYMS .tmp_vmlinux.kallsyms2.S
AS .tmp_vmlinux.kallsyms2.S
LD vmlinux
SYSMAP System.map
MODPOST Module.symvers
CC [M] fs/efivarfs/efivarfs.mod.o
CC [M] fs/nfs/flexfilelayout/nfs_layout_flexfiles.mod.o
OBJCOPY arch/riscv/boot/Image
GZIP arch/riscv/boot/Image.gz
LD [M] fs/efivarfs/efivarfs.ko
LD [M] fs/nfs/flexfilelayout/nfs_layout_flexfiles.ko
Kernel: arch/riscv/boot/Image.gz is ready
```

Figure 3.3: Result of installation of the Linux kernel

The last thing to pay attention to is that when we want to build a 32-bit RISC-V based QEMU, the Linux needs to be in 32-bit as well. Therefore, the parameter “*defconfig*” needs to be changed to “*rv32_defconfig*” when generating the *.config* file. Otherwise, QEMU will get stuck when booting Linux operating system.

3.4 The installation about busybox

Busybox, the Swiss Army Knife of Embedded Linux[20], is the most popular shell used in embedded Linux. Bruce Perence made it in 1996[21]. The initial aim for him was to create a utility for the Debian bootloader. Busybox integrates many Unix tools into one executable file, which provides 107 command functions and takes up a tiny space. Today, Busybox is part of almost all commercial Linux-based systems[21]. Busybox can be used to generate a root file system and also provide some basic programs and consoles running under Linux.

This part has four steps that need to be done. The commands to be executed are:

```
git clone https://github.com/mirror/busybox.git
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu-\
defconfig
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu-\
menuconfig
make ARCH=riscv CROSS_COMPILE=riscv64-unknown-linux-gnu-\
install
```

where the key step is the third one. It generates a text-based graphical interface shown in the figure 3.4.

What the user needs to do is to select “setting”, and then choose “Build static binary (no shared libs)”, exit and save.

A critical point to note is that the terminal interface for running this command needs to be large enough; otherwise, the graphical interface may fail to open, and an error will be reported.

3.5 Making root filesystem

When all the above tools are installed, the QEMU system-level emulation can already be executed. But there is a problem here: since the Linux kernel needs a file system, how to load the compiled Busybox and the testbench is a critical task. Hence, it is necessary to build a file system. An empty disk with a file system formatted as the second extended file system (ext2) can be made using the following commands:

```
dd if=/dev/zero of=root.bin bs=1M count=1000
mkfs.ext2 -F root.bin
```

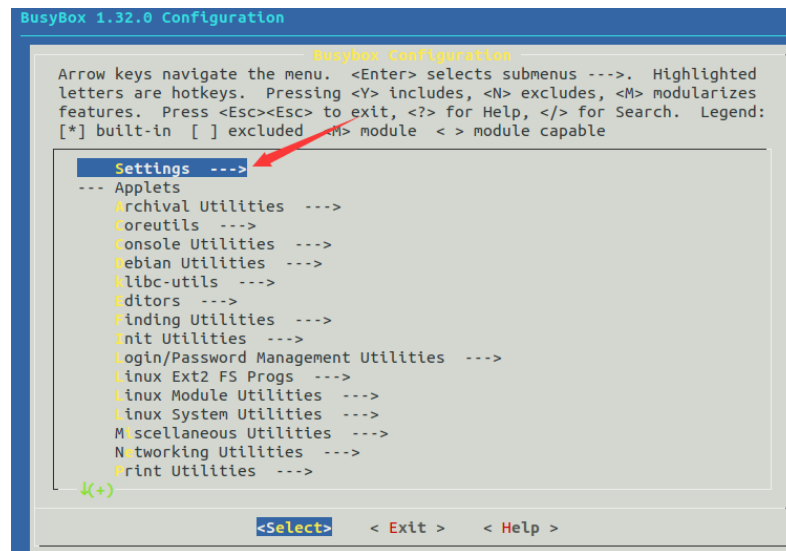


Figure 3.4: The interface about Busybox Configuration

According to these commands, an empty file system named `root.bin` with the file format `ext2` is created. Then what needs to be done is to load the Busybox into the file system. The command detail about how to load the Busybox is shown as follows.

```
mkdir mnt
sudo mount -o loop root.bin mnt
cd mnt
sudo mkdir -p bin etc dev lib proc sbin tmp usr \
usr/bin usr/lib usr/sbin
sudo cp ~/busybox/busybox bin
sudo ln -s ../bin/busybox sbin/init
sudo ln -s ../bin/busybox bin/sh
cd ..
sudo umount mnt
```

By the above command, the new file folder “*mnt*” is created and using the command “*mount*” the users can visit the file system by the new folder. Then in the new folder “*mnt*”, the new directory structure of the root filesystem is built, and the compiled Busybox can be put in the file “*bin*”, “*init*”, and “*sh*”. The compiled testbench by the compiler can be put in this filesystem using the command:

```
sudo cp ~/benchmark/benchark_1 mnt
```

This command needs to execute in the file outside the *mnt* file. To check whether the file is placed in the correct location in the root file system. The command “*tree*” can be used. The created root file system directory structure is as follows: When the root file system directory structure is the same as this

```
~/work/linux/rootfs/mnt$ tree -l
├── bin
│   ├── busybox
│   └── sh -> ../bin/busybox
├── dev
├── etc
├── lib
├── lost+found [error opening dir]
├── proc
├── sbin
│   └── init -> ../bin/busybox
├── tmp
├── usr
│   ├── bin
│   ├── lib
│   └── sbin
└── 12 directories, 3 files
```

Figure 3.5: The structure of root file system directory

figure, it has been created correctly. After that, using the command “*umount*” is necessary. If not, some newly loaded files do not appear in the directory you placed when you started QEMU with this file system.

Chapter 4

Running QEMU and compiler

4.1 Running compiler

In order for QEMU to run the written code, the users need first to compile the benchmark into a file which can execute through the compiler. Because many parameter settings are involved in the compilation, if the user inputs a very long command in the terminal, this process will become very inefficient. To improve the efficiency of compilation, a Makefile should be used so that the user only needs to enter the make command in the terminal and change some specific parameters in the Makefile to complete the compilation process. An example of the Makefile is shown as follows.

```
CROSS?=rv64imfd
ABI?=lp64d
benchmark.$(CROSS).elf: benchmark.c
riscv64-unknown-elf-gcc -march=$(CROSS) -mabi=$(ABI)
-static -o benchmark.$(CROSS).elf benchmark.c -lm -g
# riscv64-unknown-linux-gnu-gcc -march=$(CROSS)
#   -mabi=$(ABI) -static -o benchmark.$(CROSS).elf
#   benchmark.c -lm -g
clean:
-rm -f *.elf *.o
```

According to the above code in the Makefile, the Makefile is divided into two parts. The first part is the compilation part, where

- The first two lines is a definition which defines a specific RISC-V instruction set combination and corresponding application binary interface (ABI), and the third line states the output and input files.

- The “*riscv64-unknown-elf-gcc*” means the first mode of the compiler is chosen and this compiler will compile the C program. The detail about the modes of the compiler has been introduced in section 3.1.4.
- The “*-march*” and “*-mabi*” are a specific RISC-V instruction set combinations and a corresponding application binary interface (ABI) relatively. The combination of the RISC-V instruction sets in this thesis are introduced in chapter 1.
- The parameter “*-static*” means this compilation process is used as a static library. The successfully compiled executable file can run independently without requiring external requirements to read the contents of the function library when the static library is used.
- The parameter “*-o*” is followed by the output file.
- The parameter “*-l*” is used to specify the library to be linked by the program, and “*-lm*” means the program links to the math library. These parameters must be put after the input file.
- The parameter “*-g*” means that the generated executed file includes the information about debugging. It is used when the user wants to use a debugger to check the program. For the bare-metal test involved later, this parameter is required.

Then for part two, it is used to clean executed files. By inputting the command “*make clean*”, the file whose format is **.elf* or **.o* will be deleted.

The way to compile the file using another mode of the compiler is also shown but commented out. This Makefile cannot run two compilers at the same time. When the user switches modes, he or she only needs to uncomment the second one and comment on the other one.

When this Makefile is created, what the user needs to do is just to change the name of the input and output. Then they choose the specific RISC-V instruction set combination and corresponding application binary interface (ABI). In this way, all programs can be compiled efficiently.

4.2 Running QEMU

As mentioned above, the QEMU has two modes: user mode and system mode. For running QEMU in user mode, it is simple. When the QEMU is installed

according to chapter 3, it has been added as a global variable. The user only needs to input the command:

```
qemu-riscv64 ./benchmark.elf
```

Then about system-mode for the QEMU, it has many parameters that can be set. The specific commands are shown below:

```
qemu-system-riscv64 -nographic -machine virt \
-kernel linux/arch/riscv/boot/Image \
-append "root=/dev/vda rw console=ttyS0" \
-drive file=rootfs/rootfs.img,format=raw,id=hd0 \
-device virtio-blk-device,drive=hd0 \
-bios default
```

According to the above command,

- The parameter “**-nographic**” means QEMU is set in no display GUI mode. It will not show a specific development board model.
- The parameter “**-machine**” is used to choose the platform and using the parameter “**-machine help**” can view the platform information supported by the QEMU.
- The parameter “**-kernel**” is used to provide the Linux kernel.
- The parameter “**-append**” provides the kernel command line arguments and the “**console=ttyS0**” allows the input and output of QEMU display on the current terminal without jumping to a new terminal.
- The parameter “**-drive**” is used to add image files, which are generally used for block devices. Here it is used to specify the path, format, and id of the filesystem image.
- The “**-device**” is often used to identify external devices mounted on the bus of the guest, such as Universal Serial Bus(USB), Peripheral Component Interconnect(PCI) and other buses. Here the “**virtio-blk-device**” is an implementation of virtual disks.
- The mode of the bios is set as default.

After inputting this command, the QEMU can be run perfectly, and the user can use it just like a standard Linux system. The result of this command is shown in figure 4.1. And the user can press enter to activate the console.

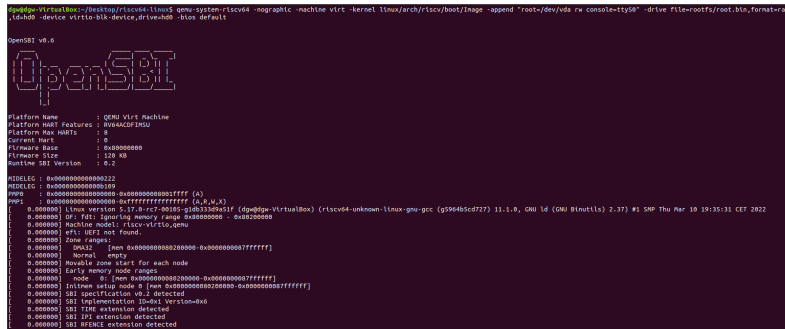


Figure 4.1: The initial interface of QEMU

After activating the console, the Linux commands might not work as shown in figure 4.2. This is because the Linux system has not started the Busybox which consists of many Linux basic commands. And by inputting “*/bin/busybox –install -s*”, the Busybox can be started. Then by the command “*uname -a*”, the user can see that the RISCV64 Linux is running normally. Then the benchmark loaded from the file system can be run in the QEMU where the command “*time*” can be used for recording the benchmarks’ running time.



Figure 4.2: The console of Linux kernel on QEMU

Finally, by alt+A+C, the user can exit QEMU in the terminal.

Chapter 5

Benchmark

A total of ten different benchmarks are used to test the performance of QEMU in this chapter. Each type of benchmarks has its unique focal point for testing. For example, some benchmarks only operate on variables whose data type is an integer, while others work on data whose data type is a floating-point number. This difference makes QEMU's runtime distinguish when using the different RISC-V instruction sets. The list of the ten benchmarks is shown in table 5.1:

Benchmark			
Name	Source file type	Necessary ISA	File input/output
Fibonacci sequence	C	Integer(I)	No input/output file
Dhrystone	C	Integer(I)	No input/output file
Linpack	C	Integer(I),Multiply(M),and Floating-point(FD)	No input/output file
Speed	C	Integer(I),Multiply(M)	No input/output file
Matrix multiplication	C	Integer(I),Multiply(M)	No input/output file
Signal filter	C	Integer(I),Multiply(M)	No input/output file
Simple neural network	C	Integer(I),Multiply(M),and Floating-point(FD)	Input file needed, no output file
Image process	C	Integer(I),Multiply(M)	Input and output file needed
Fast Fourier transform	CPP	Integer(I),Multiply(M),and Floating-point(FD)	No input/output file
Data compression	CPP	Integer(I)	Input file needed, no output file

Table 5.1: The list of benchmarks

All of them are written in C or C++. These ten benchmarks used in this paper will be introduced in the rest of this chapter.

5.1 Fibonacci sequence

As one of the classic mathematical models, the Fibonacci sequence was discovered by the mathematician Leonardo Fibonacci.[22] Specifically, it refers to a sequence like this 1, 1, 2, 3, 5, 8, 13, 21, 34... The sequence starts at term 3, and each term is equal to the sum of the previous two. And in

mathematics, the Fibonacci sequence is recursively represented as:

$$\begin{aligned} F(0) &= 0, \\ F(1) &= 1, \\ F(n) &= F(n-1) + F(n-2) (n \geq 2, n \in N^*). \end{aligned} \quad (5.1)$$

It is defined as a critical number sequence used in modern physical chemistry and other fields.

In this study, the Fibonacci benchmark is written with C, and the main data type is in integer formats. Because it is an infinite sequence, users can decide how many numbers should be generated to control the amount of computation of QEMU. According to the mathematical formula of the Fibonacci sequence, it is easy to know that its main operation method is addition and only focuses on integers. Hence, for the strategies of designing the program of the Fibonacci sequence, there are only two parts. One is the function block which aims to realise the function of the Fibonacci sequence, and another is the main part which seeks to call the function module and set the number of the sequence generated.

The code of the function part is:

```
static int fib(int i) {
    return (i>1) ? fib(i-1) + fib(i-2) : i;
}
```

It means that when i is more significant than one, the return value is equal to the sum of the two numbers before it and i when i is smaller than one. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8.

5.2 Dhrystone

Dhrystone is a very popular and classical benchmark which is created by Dr Reinhold P. Weicker from Siemens AG in 1984.[\[23\]](#) The initial aim of this benchmark is to create a brief benchmark program that can represent a whole system. It includes some integer arithmetic and operation about strings and logic, which tries to simulate the activities of the CPU. [\[24\]](#) According to the source file of the Dhrystone, the main components of this benchmark are assignments, control statements, procedures and function calls. And 8 procedures are called in one iteration. But the Dhrystone is not suitable for modern computer architectures. Since the hardware situations

have a considerable progress in the last decades and the requirements for the corresponding programs also changed, more advanced algorithms are required to meet today's demands. For modern processors, the code size of Dhrystone is so small that it cannot accurately reflect the processor performance as before. But for an embedded system, it has enough computational scale to serve as a benchmark.

This benchmark is written in C, and the main data type is integer. The core algorithm is to make a loop consisting of several programs and run this loop many times to obtain the accumulative computation time. The specific process is shown in figure 5.1:

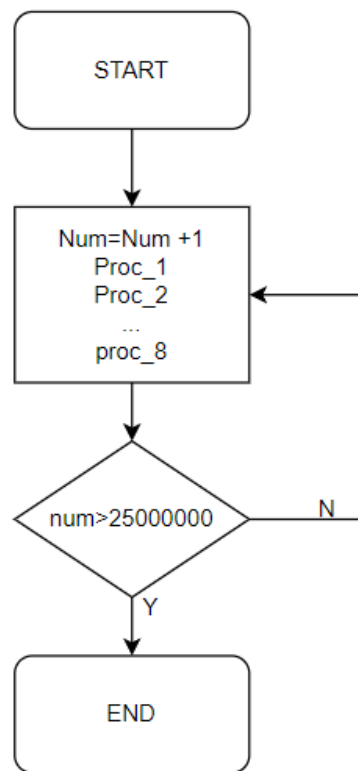


Figure 5.1: The logical block diagram about Dhrystone

According to the figure about the Dhrystone, it has eight procedures included, and the number of iterations executed is 25000000. The analysis of the result is shown in chapter 8.

5.3 Linpack

Linpack is another widely used benchmark in the world. Its main task is to test the floating-point performance of a high-performance computer system. Same as the Dhystone, it was born by accident. Its original aim was to provide some information to help the user know the execution time for solving a linear equation system[25]. Over time, people added more functional tests to it and eventually formed a widely used benchmark. Nowadays, the benchmark Linpack tests the floating-point computing performance of a high-performance computer by solving the one-dimensional N-th dense linear algebraic equation system using the Gaussian elimination method[26]. This benchmark includes three categories, Linpack100, Linpack1000 and HPL. The first one solves the group of the linear equations of order 100, the second one solves that of order 1000, and the HPL means high-performance Linpack, called a highly parallel computing benchmark[26]. This thesis uses the first one. Same as the Dhystone, the Linpack100 is not suitable for modern computing but for embedded systems. It qualifies as a benchmark.

This benchmark is written in C, and the main data type is float and integer. The core algorithm is to solve a group of linear equations. There are three different main functions which are “*matgen*”, “*dgefa*”, and “*dgesl*”. The first function, “*matgen*”, is used to generate different kinds of matrices used in the following functions. Then the function “*dgefa*” is aimed to do LU factorisation for the matrix, which is generated from the function “*matgen*”. The output of this function is used as the input of the function “*dgesl*”, which can get the solution of the linear equations.

The specific process of the benchmark is to execute the three functions in sequence multiple times. And after completing the combination of these three functions for the first time, this benchmark uses the residual to verify the correctness of the results and then executes these three functions in a loop. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8. That chapter will analyse the results in detail.

5.4 Speed

Various basic mathematical operations also need to be specially tested. This benchmark is mainly for integer operations, including the main operation types of addition, subtraction, multiplication and division. The main structure is shown in the figure 5.2

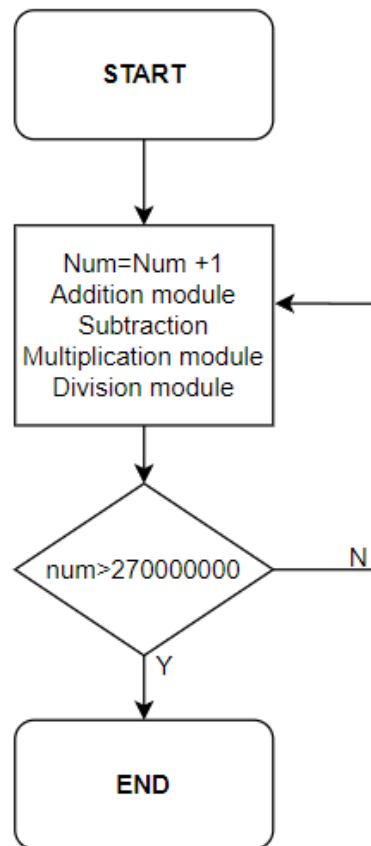


Figure 5.2: The logical block diagram about speed

According to figure 5.2, this structure is the same as the benchmark Dhrystone. Its main part is divided into four modules: addition, subtraction, multiplication and division. Addition adds a set of eight other variables and asks them to add an immediate number. After that, the inverse operation is performed in the subtraction part. The same goes for the multiplication and division parts. Afterwards, in order to make the test results more obvious, these four modules are put into a loop and the number of loops is set as 270000000. This benchmark is written in C. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8.

5.5 Matrix multiplication

Matrix multiplication, is mainly used as a benchmark to realise the multiplication of two large matrices. The data used in this part are mostly in integer types, and the operation type involved is multiplication between vectors. There are also specific regulations when doing matrix multiplication. An $m \times n$ matrix is an array of $m \times n$ numbers arranged in m rows and n columns. It only makes sense if the number of columns of the first matrix and the number of rows of the second matrix are the same. The specific principle of matrix operation is shown in the following equation.

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nm} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1m} \\ c_{21} & c_{22} & \dots & c_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ c_{m1} & c_{m2} & \dots & c_{mm} \end{bmatrix} \quad (5.2)$$

According to this equation, the number of rows in the result matrix is equal to the number of rows of the first matrix, and the number of columns of the result matrix is similar to the number of columns of the second matrix. The element of the m_{th} row and the n_{th} column of the product C is equal to the sum of the products of the element of the m_{th} row of the matrix A and the corresponding element of the n_{th} column of the matrix B . In order not to make the dimension of the matrix problematic in this thesis, the size of the first and second matrices are 800×800 .

This benchmark is written in C. In addition to initialising some arrays and variables, the main part consists of the matrix generation and multiplication function. About the matrix generation, this function uses the function “*rand()*” to generate the element of the matrix with the range of 10000. And then about the function of matrix multiplication, its duty is to get the result about the matrix multiplication according to the regulation for matrix multiplication mentioned in the previous paragraph. By looping and accumulating, the outcome of the matrix multiplication will be obtained. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8. That chapter will analyse the result in detail.

5.6 Signal filter

This part aims to test the digital signal processing performance of QEMU. In digital signal processing, the filter plays an essential role. It can remove

noise from the received signal. Furthermore, the Finite Impulse Response (FIR) signal filter is fundamental in the signal filter family. It can be called a non-recursive filter as well. It has strict linear phase-frequency characteristics while ensuring arbitrary amplitude-frequency characteristics, and its unit sampling response is finite, so the filter is a stable system. Hence it is widely used in communication, figure processing, etc. In this thesis, the FIR filter model is chosen as a benchmark to represent a signal process. Furthermore, the direct-form structure and equation of the FIR filter are shown as follows:

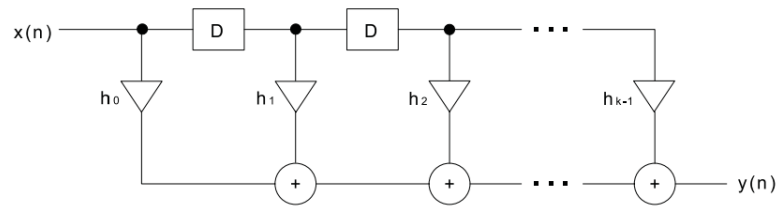


Figure 5.3: The direct-form structure of the FIR filter[27]

The equation is on the form:

$$\begin{aligned}
 y[n] &= b_0x[n] + b_1x[n-1] + \cdots + b_Nx[n-N] \\
 &= \sum_{i=0}^N b_i \cdot x[n-i]
 \end{aligned} \tag{5.3}$$

where

$y[n]$ is the output of the signal;

$x[n]$ is the input of the signal;

$N-1$ is the filter order;

b_i is the influencing factor of the filter.

According to the math model, the algorithm written in the C programming language is:

```

void fir(short x[], short b[], short y[])
{
    int i, j; long long sum;
    for (j = 0; j < n; j++)
    {
        sum = 0;
        for (i = 0; i <= N; i++)
            sum += x[j-i] * b[i]; y[j] = sum;
    }
}

```

This benchmark calls this function in the main part to deal with signal data given in the main part. Furthermore, this benchmark is written in C, and the primary type of operation is addition and multiplication. Moreover, the result of this benchmark based on different RISC-V ISA is shown in Chapter 8.

5.7 Neural Networks

In recent years, the neural network has been a prevalent mathematical or computational model. The algorithm evaluates or approximates the function by imitating the structure and function of the biological neural network to achieve the desired result. Today it has applications in image and video, medical games, and robotics[28]. Therefore, it is essential to run the model through the embedded system. This benchmark is aimed to test the performance of QEMU in running a Neural Network.

A neural network consists of a large number of connected nodes. Each node represents a specific output function, called the activation function. The connection between every two nodes represents a weighted value for the signal passing through the connection, reaching the weight, which is equivalent to the memory of the artificial neural network. After the model is repeatedly trained through many training data sets, the output results are obtained through the test data input data and compared with the correct results to test the model's accuracy.

This benchmark is written in the C language, and it uses a three-layer structure of the neural network model, the input layer, the hidden layer and the output layer. There are three nodes in the input layer, five nodes in the hidden layer, and one node in the output layer. A random function generates the value of the weight. The training and test data are imported through external files (the format is .txt). Like a general neural network, this benchmark uses training set to optimize the weights of the neural network and then uses the test set to generate test results. Then accuracy score of the model can be derived from the comparisons with the ground truth. The structure diagram of the model is shown in figure 5.4.

The main type of operation in this benchmark is addition and multiplication, and the data type includes integer and float. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8 that analyses the result in detail.

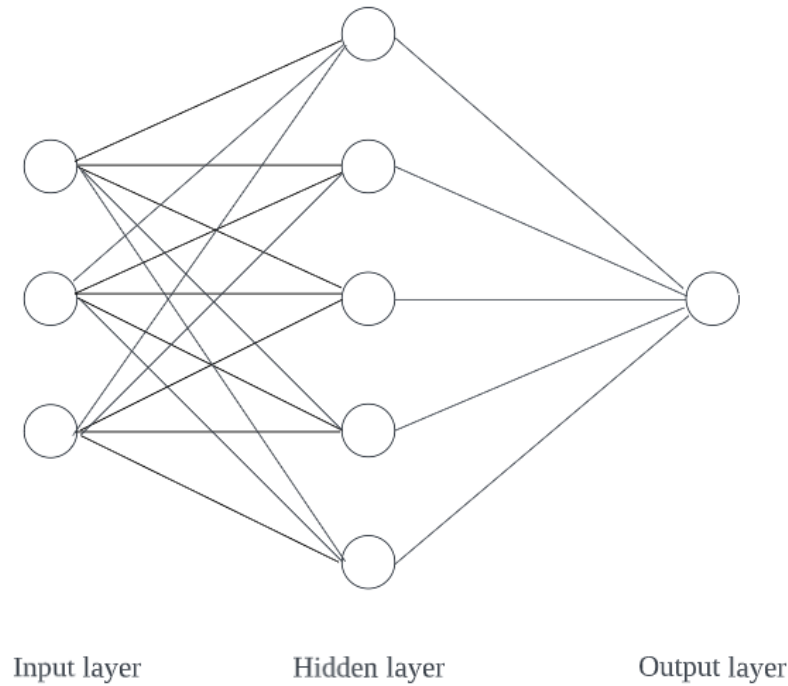


Figure 5.4: The structure of the Neural network

5.8 Imageproc

With the development of technology, it seems to be more and more common to use a smartphone to take pictures and using enhanced image processing. It becomes more and more demanding on pushing the performances of algorithms when people try to analyse and process the collected images, such as image compression, enhancement and restoration, matching, description and identification[29]. The research on image processing algorithms has been developed very well, and the functions that can be realised are becoming more and more powerful. Therefore, it is also necessary to use image processing as a general benchmark to test the performance of QEMU. However, the mature image processing algorithms developed today has become very complex. This benchmark only covers functions such as image grey-scale conversion and image sharpening.

Image processing often starts from the digitization with certain techniques [30]. Digital images refer to a large two-dimensional array obtained by shooting with industrial cameras, video cameras, scanners and other equipment. The elements of the array are called pixels, and their values are

called gray values. Image grayscale conversion refers to converting a colour image to a grayscale image. The colour of each pixel in the colour image is determined by the three components of R(Red), G(Green), and B(Bule), and each component can take a value from 0 to 255. Therefore, a pixel can have more than 16 million ($255 \times 255 \times 255 = 16581375$) range of colour changes. The grayscale image is a particular colour image with the same three components of R, G, and B. The transformation range of one pixel is 256, so in digital image processing, images of various formats are generally converted to grayscale. The description of grayscale images, like colour images, still reflects the distribution and characteristics of the overall and local chromaticity and highlights levels of the entire image.

There are many methods to realise this function, such as the Average algorithm, Maximum algorithm and Weighted average algorithm. About the weights of R, G, and B, they take different values to form different grayscale images. According to Luminance [31], since the human eye is most sensitive to green, followed by red, and the least sensitive to blue, a grayscale image that is easier to identify will be obtained. Generally, the obtained grayscale image is the best. Moreover, the weight occupied by the three colours is shown in the following formula:

$$\text{Gray} = (\text{Red} \times 0.3 + \text{Green} \times 0.59 + \text{Blue} \times 0.11) \quad (5.4)$$

According to this weight distribution, the input and output pictures are shown in the figure 5.5 and figure 5.6:



Figure 5.5: The input picture of Figure 5.6: The output picture of grayscale conversion

Image sharpening compensates for the image's contour, enhances the edge of the image and the part of the grayscale transition, and makes the picture clear. It is divided into two categories: spatial domain processing

and frequency domain processing. Image sharpening highlights the edges, contours, or features of some linear target features on the image. This filtering method improves the contrast between the edge of the feature and the surrounding pixels, which is also called edge enhancement.

The edges of objects appear in the form of discontinuities in the local properties of the image, i.e. edges signify the end of one region and the beginning of another. Image edges have two parameters, direction and magnitude. Typically, pixels run along the edge very gently, while pixels run perpendicularly to the edge vary drastically. Hence, finding an object's edge is finding the amount of data change in a certain direction, which can be achieved by gradient calculation. The pixel's grayscale varies significantly at the object's edge, and the gradient value is enormous. In the non-edge region, the gradient value is minimal.

The method used in this paper is the Sobel operator method in the differential method, and its specific budget form is as follows:

Assume that matrix A comprises an original pixel and eight adjacent pixels around it. Convoluting with the shown convolution kernels in the horizontal and vertical directions, respectively.

$$\begin{aligned} G_x &= \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \cdot A \\ G_y &= \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \cdot A \end{aligned} \quad (5.5)$$

At each point of the image, combine the above two results to find the approximate gradient by

$$G = \sqrt{|G_x| + |G_y|} \quad (5.6)$$

A threshold might be set to determine whether the current pixel is on the object's edge. A sharpening effect is applied to the current pixel if the gradient is greater than the threshold. The pixel in question is not processed if the gradient does not go over the threshold, leaving the image outside of the edges unaltered. Furthermore, the input and the output image are shown as follows:



Figure 5.7: The input picture of Figure 5.8: The output picture of image sharpening

This benchmark is written in C and the main type of operation is addition and multiplication. And its input image format must be *.bmp*. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8, which analyses the results in detail.

5.9 Fast Fourier transform

The Fast Fourier transform is a general term for efficient and fast computing methods for computing discrete Fourier transform (DFT) using a computer, referred to as FFT. The Fast Fourier Transform was proposed in 1965 by J.W. Cooley and T.W. Tukey [31]. This algorithm plays a crucial role in the digital signal processing process. It is frequently used in the communication industry. Hence, this benchmark aims to evaluate how well the QEMU performs when executing a quick Fourier transform algorithm.

Radix-2 Cooley-Tukey is the FFT algorithm used in this benchmark [31]. Cooley-Tukey Fast Fourier Transform (FFT) algorithm is a ubiquitous algorithm to accelerate discrete Fourier transform (DFT), a discrete form of the Fourier Transform. The benchmark starts with the creation of a spectrum using femtosecond laser pulses' electromagnetic fields. On the spectrum, an FFT and an IFFT are then run. Internal functions check the accuracy of the outcome following each procedure.

This benchmark is written in C++, including the integer and float data types, and many operation types are included. About the result of this benchmark based on different RISC-V ISA is shown in Chapter 8.

5.10 Compression

Data storage is significant for computers, but the data generated after the operation will become very large. It is inconvenient to do data transmission and storage. So it is vital to use a suitable compression algorithm. This algorithm can save a lot of storage space to prevent data explosions and computer crashes. Therefore, it is also necessary to use a suitable compression algorithm for the established virtual platform. This benchmark is used to test the performance of the compression algorithm running on QEMU. A total of three algorithms are mentioned in this part.

Firstly, the algorithm is called Run Length Encoding (RLE). It is a simple compression algorithm. The core idea of the algorithm is to calculate the number of consecutive repeated occurrences of a series of consecutive English letters and then use the corresponding single character and the number of occurrences to re-count. In the following example, the data is *AAAAAABBCDDEEEEF*, which occupies a total of 17 characters. Furthermore, with this algorithm, it can be represented as *A6B2C1D2E5F1*. It is easy to find that many of the characters appear continuously and repeatedly. The user can first calculate the number of occurrences and then mark them. Its advantage is that the algorithm is straightforward, but it can only be effective for some specific sequence data, and some data will have a higher data space occupancy rate after using this algorithm to compress.

Secondly, the algorithm is called Lempel-Ziv-Welch coding (LZW). The LZW algorithm adopts the method of dynamically establishing a dictionary, and reads the character sequence of the original file in turn. When a new continuous string is encountered, a mark is added to the dictionary. Directly replace the string with the dictionary index number and write it into the compressed file. Two nouns are introduced here: "string", and "char"; string represents the prefix, char represents the newly read character, and each dictionary index corresponds to a pair (string, char); Take the string sequence *ABCABC* as an example,

1. Firstly, the characters A, B, and C are recorded in dictionary indexes 1,2,3.
2. At the beginning, read the first character string = A, read the next character char = B;
3. Look up the dictionary (A, B); if it is not found in the dictionary, record (A, B) in the dictionary index 4, then output the prefix A as 1, update string=B, reread the characters, char=C;

4. Look up the dictionary (B, C); if it is not found in the dictionary, record (B, C) in the dictionary index 5, then output the prefix B as 2, update string=C, reread the character, char=A;
5. Look up the dictionary (C, A); if it is not found in the dictionary, record (C, A) in the dictionary index 6, then output the prefix C as 3, update string=A, reread the character, char=B;
6. Look up the dictionary (A, B), the dictionary can be found, corresponding to the index number 4, update string=AB, reread the characters, char=C;
7. Look up the dictionary (AB, C); it is not found in the dictionary, record (AB, C) in the dictionary index 7, then output the prefix AB as 4, update string=char=C, reread the characters, char=NULL;
8. char = NULL, end of file, output prefix C.

Furthermore, the output is represented as **12343**, which is shorter than the original string sequence. When the string is long enough, the compression efficiency of this algorithm will become higher and higher. It is very similar to the first compression algorithm's principle but more efficient.

Thirdly, this algorithm is called Huffman coding, also a lossless compression algorithm. However, unlike the LZW compression algorithm, Huffman requires prior knowledge of the occurrence probability of each character. The purpose of compression is achieved by calculating the frequency of occurrence of each character in the character sequence and carrying out a unique coding design for each character so that the character with highest frequency occupies the shortest number of digits. In contrast, the character with low frequency is long. Usually, 20% 90% of space can be saved.

For a string: **AABCDACA**, where the frequency of A is 4, the frequency of C is 2, and the frequency of B and D is one, respectively. According to the frequency of each word, draw a Huffman tree like the following. It starts with the minor frequency, such as B, D, which appears once, and the two add up to 2, so their previous node is 2. This graph has been constructed in this way.

According to this figure, A is **1**, B is **000**, C is **01**, and D is **001**. Furthermore, the string can be represented as **1 1 000 01 1 001 1 01 1**, which will occupy 15 bits after compression, and the original string occupies $8 \times 8 = 64$ bits. Hence, this algorithm has a very high compression ratio.

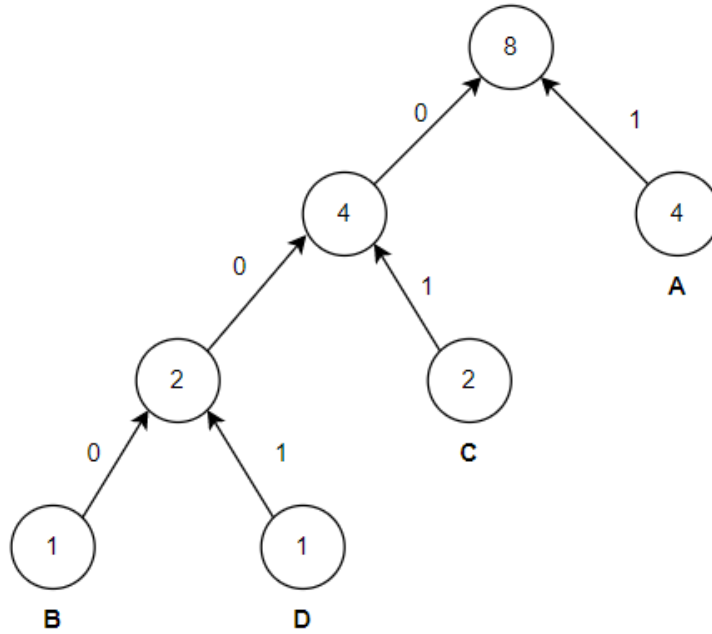


Figure 5.9: The structure of Huffman tree

This benchmark includes the three different compression algorithms but only runs the third compression algorithm to test the performance of QEMU. Furthermore, the input data is a famous figure, “Lena”, which is shown as follows and is represented as a vast array and stored in the head file. It is written in C++, and the main data type is integer. The result of this benchmark based on different RISC-V ISA is shown in Chapter 8.



Figure 5.10: The test figure “Lena”

Chapter 6

“Virt” platform analysis

For running QEMU on system mode, it is vital to choose the desired platform. It will determine what kind of computer architecture the programs will run on. When running the QEMU, a parameter called the *machine* is used to choose the type of board. However, for the QEMU based on RISC-V processor, many platforms based on RISC-V are not included in the platform library by default. There are a total of four types of platforms that can be used. These platforms are **Microchip PolarFire SoC Icicle Kit (microchip-icicle-kit)**, **Shakti C Reference Platform (shakti_c)**, **SiFive HiFive Unleashed (sifive_u)**, and **Generic Virtual Platform (virt)**. The Specific parameters about these platforms can be obtained through the command “*-machine help*” when running the QEMU. Models for these platforms are written in C, which can be found in the folder of “*hw/riscv*” of the downloaded QEMU. And users can add their platform source code in the platform library. However, it is going to be a vast amount of work. Writing a fully functional platform in C language is difficult for a beginner. At the same time, after putting the completed platform model into the folder “*/riscv*”, many QEMU configuration files need to be modified to make QEMU compatible with newly added platform models.

Therefore, this thesis uses the “*virt*” as the test platform. The platform’s full name is the QEMU RISC-V VirtIO board, which the Company SiFive designs. There is no actual platforms model. It is specially designed for use in virtual machines. In general, this platform needs a suitable Linux system as an operating system. It supports Peripheral Component Interconnect(PCI), Virtio and CPUs and Random-access memory(RAM).

The rest of this chapter will focus on three essential parts of the platform’s source code: the memory space distribution of the “*virt*” platform, the “*virt*” platform startup process, and the loading of firmware and kernel.

6.1 Memory space distribution

After reading the source code about the “*virt*” platform, the memory space distribution is set in the function “*virt_memmap()*”. Its specific distribution is shown in the following table.

Memory space distribution		
Start address	End address	Definition
0x0	0x100	VIRT_DEBUG
0x1000	0x100000	VIRT_MROM
0x100000	0x101000	VIRT_TEST
0x101000	0x102000	VIRT_RTC
0x2000000	0x2010000	VIRT_CLINT
0x3000000	0x3010000	VIRT_PCIE_PIO
0xc000000	0x10000000	VIRT_PLIC
0x10000000	0x10000100	VIRT_UART0
0x10001000	0x10002000	VIRT_VIRTIO
0x20000000	0x24000000	VIRT_FLASH
0x30000000	0x40000000	VIRT_PCIE_ECAM
0x30000000	0x40000000	VIRT_PCIE_ECAM
0x40000000	0x80000000	VIRT_PCIE_MMIO
0x80000000	0x88000000	VIRT_DRAM

Table 6.1: Memory space distribution of “virt” platform

According to the table 6.1, it is easy to know the memory space distribution of the “*virt*” platform. Furthermore, about the part of Dynamic random access memory (DRAM), its default size is 0x8000000, 128M. Nevertheless, when starting QEMU, a parameter called “*-m*” can be used to set the DRAM size. It can set more than 128M.

6.2 Starting process of “virt” platform

About the starting process of this platform, this platform will access the memory block named VIRT_MROM and then complete the module’s initialization through the function “*memory_region_init_rom*” and by the function “*memory_region_add_subregion*” Mask read-only memory (MROM) has been defined as a sub-region of system memory.

After that, the data is stored in the MROM through a function “*riscv_setup_rom_reset_vec*” defined in the file “*boot.c*”. This function

executes a piece of binary instruction data compiled from assembly code. The source code is put in the Appendix. What it accomplishes is as shown in the comments. Its main task is to jump to the base address of the DRAM where the firmware or kernel program is stored. It means that the operating system will be started.

6.3 The loading about the firmware and kernel

In the source code “*virt.c*”, the firmware is loaded by the function “*riscv_find_and_load_firmware*” which is defined in the file “*boot.c*”. This function provides three different situations to load the firmware. When running the QEMU, a parameter called “*bios*” can be set, which decides the situation about loading firmware. These three situations are as follows.

1. The built Open Supervisor Binary Interface(OpenSBI) in the QEMU is loaded as firmware when the parameter “*bios*” is not set or set as default.
2. The firmware is not loaded when the parameter “*bios*” is set as “none”.
3. User can load their own firmware by setting the parameter “*bios*” as the file name from local.

The format of the firmware loading file is ELF or normal binary format. Files in ELF format have higher priority than normal binary format files. Similarly, about the loading of the kernel, the format of the kernel includes ELF format, uImage format, and normal binary format. Their priority is ELF format higher than uImage format than normal binary format.

Finally, the default base address of the firmware is the start address of the DRAM. It is 0x80000000. But if the firmware is loaded by a file whose format is ELF, the base address depends on the section table information of ELF, instead of the default base address of the DRAM. Similarly to the firmware, the base address of the kernel is 0x80200000 for the platform of RISC-V 64 and 0x80400000 for the platform of RISC-V 32. And if the kernel is loaded by the ELF file, the base address of the kernel is determined by the section table information of the ELF. Furthermore, when loading the ELF format and encountering a memory layout conflict, QEMU will output an error message and exit.

Chapter 7

The bare-metal test on QEMU

This chapter mainly explores the possibility of running benchmarks on QEMU under bare metal conditions. The so-called bare metal refers to the absence of an operating system. When an operating system exists, the time taken from the test to running the benchmark includes the time taken to start the operating system, which may cause a certain deviation in the test results. Therefore, it is necessary to study the possibility of QEMU running the benchmark in the case of bare metal.

However, in the official documents, there are no guides for the bare metal test of QEMU-RISCV. There is only an introduction to bare-metal programming in the official documentation for QEMU-ARM. It mainly introduces that under the ARM architecture, the base address of the platform “*virt*” DRAM is 0x4000_0000. For guests booting as “*bare-metal*,” the program takes 0x4000_0000 as the start address and loads the compiled benchmark through the parameter “*-kernel*” which tells QEMU what to run[32]. In the previous chapter, which introduces the platform “*virt*”, it is easy to know that the base address of the DRAM in the “*virt*” platform under the riscv structure is 0x8000_0000. Besides, a user can add the parameter “*-machine dumpdtb=<file>*” to get the information about the device tree. Furthermore, the specific instructions to be executed are as follows:

```
$ qemu-system-riscv64 -machine virt, \
    dumpdtb=qemu-riscv64-virt.dtb
$ dtc qemu-riscv64-virt.dtb > qemu-riscv64-virt.dts
```

After inputting these commands in the terminal, a file named “*qemu-riscv64-virt.dts*” will appear, and it can provide information about the device. Furthermore, the information about memory in this file is shown as follows.

```
memory@80000000 {
    device_type = "memory";
    reg = <0x00 0x80000000 0x00 0x80000000>;
};
```

This information means that the memory ranges from 0x8000_0000 to 0x8800_0000. This message is the same as mentioned in the previous chapter.

Therefore, two ideas might be implemented in this process to load the benchmarks into QEMU. For the first idea, the user can modify the layout of the memory space of the “*virt*” platform by changing the based address of RAM to 0x0000_0000 so that the executed binary file compiled by the compiler can load the benchmark program. Nevertheless, using this method is unreasonable. As chapter 6 says, the platform is written by C, it is easy to modify, but the algorithms used are complex. Often modifying one place will cause unknown bugs in other parts. Furthermore, the range of the RAM is dynamic. It can be set when starting QEMU. It will affect the allocation of other devices on the platform. Such as if the base address of RAM is set as 0x0000_0000. It will affect the boot of the ROM so that the whole QEMU cannot start.

Another idea is to modify the compilation process’s parameter so that the executed binary file, whose formal is ELF, can set the 0x8000_0000 as the initial address. In order to achieve this purpose, on the one hand, the compiler can only choose “*riscv-unknown-elf*” because another compiler is for a Linux system. The compiler’s output file can only run on the Linux system. On the other hand, a linker script is needed, which main purpose is to manage the memory layout of the output file and describe how to convert the input file to the output file. Users can use the default linker script and modify part of it as the new linker script. Its specific commands are as follows:

```
$ riscv64-unknown-elf-ld --verbose \
    > qemu-riscv64-virt.ld
```

After inputting these commands in the terminal, the new linker script will appear in the current folder then the user can open the file and modify the part of starting address of the memory, which is shown as follows:

```
PROVIDE (__executable_start = SEGMENT_START
("text-segment", 0x10000)); . = SEGMENT_START
("text-segment", 0x10000) + SIZEOF_HEADERS;
```

After modifying the address of the memory, save it as a new file and complete the compilation by adding “*-Wl,-T <file>*” when compiling.

However, this method is only suitable for the based address is less than 0x8000_0000. When the address is larger than 0x8000_0000, some errors will appear, and according to the error information, the problem happens in the compilation process called “*collect2*”. This process’s main purpose is to initialize some functions and pass along arguments such as *crt0.o*, *crtbegin.o*, and *crtend.o*. And in these parameters, the object “*crt0*” is the most important one. It includes a symbol called “*_start*” which can initialize the execution of the program. Because the address is larger than 0x8000_0000, the default “*crt0*” is not suitable. Therefore, the user must self-recreate a *crt0.s* file to allocate the memory address during the compilation process.

The specific content of the *crt0.s* file has been placed in Appendix 2. It is an assembly file. In this file, a variable “*_start*” is defined, which calls the function *main()*. Then the instruction “*la*” which means load address, is used to load the variable “*__executed_start*”. This way, the compiled file’s initial address can be set after 0x8000_0000. Hence, the compilation commands for the bare metal state are as follows:

```
riscv64-unknown-elf-gcc -g -Wl,--gc-sections \
    -nostartfiles s -Wl,-T,riscv64-virt.ld \
    crt0.s benchmark.c
```

where

- the parameter “*-Wl*” is a comma-separated list of flags to pass on to the linker (*ld*).
- “*--gc-sections*” stands for “garbage collect sections” and tells *ld* to remove unused sections post-link.
- “*-nostartfiles*” tells the linker not to link in any standard system startup files (such as the default *crt0.s*).
- “*-T*” allows the user to specify the path to your linker script.

In this way, the actual address of the output compiled file can be set anywhere. In order to prevent conflicts with firmware loading, this article sets the actual address of memory at 0x8200_0000.

After solving the problem about the start address, it is time to run the benchmark on QEMU. A debugger named *gdb* is used as a test tool. Its full name is “**GNU symbolic debugger**”. It is a very commonly used program debugger. Its main functions are:

1. When the program starts, you can run the program according to the customized requirements.
2. The debugged program can be suspended in the specified code.
3. You can view the current status of the program.
4. The value of some variables or the executed code's order can be changed during execution.

GDB is installed when the compiler is installed. Before starting gdb, you need to start QEMU through the following command:

```
qemu-system-riscv64 -machine virt \
-m 128M -gdb tcp::1234 -S \
-kernel benchmark.elf -bios default
```

Where the parameter **"-kernel"** is used to load the benchmark and the **"-gdb tcp::1234 -S"** is used to start the gdb server. After QEMU starts, it waits for a connection to be established with gdb. When the connection is established, QEMU will continue running its RAM's contents immediately. This parameter can be abbreviated as **"-s -S"**.

After inputting the above command in the terminal, use another terminal to boot the gdb by the command:

```
riscv64-unknown-elf-gdb ./benchmark.elf
```

Then after entering the debugging interface, the user can input the command **"target remote:1234"** in the terminal to build a connection between the gdb and QEMU.

The interface after gdb is started shown in the figure 7.1. There are many commands gdb used. The commands used in this thesis are **"break"**, **"continue"**, **"step"**, **"jump"** and **"info"**. The command **b(reak)** means setting a breakpoint and ensuring the kernel can pause in the preset position. The break instruction shown in the figure 7.1 is to set the breakpoint at the beginning of the main function. The command **c(ontinue)** means to continue the program. The **"continue"** instruction shown in the figure 7.1 will run the instructions in the RAM until the start address of the main part. Because the way of loading the firmware is set as default when QEMU starts, the range from 0x8000_0000 to 0x8200_0000 is used to load the firmware. Therefore, after inputting the command **"c"**, the QEMU firmware loaded message successfully will appear in another terminal, as is shown in the figure 7.2. It means the QEMU is still running.

```
dgg@dgg-VirtualBox:~/Desktop/riscv64-linux$ riscv64-unknown-elf-gdb ./dhrystone_bn.elf
GNU gdb (GDB) 10.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pc-linux-gnu --target=riscv64-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./dhrystone_bn.elf...
(gdb) target remote:1234
Remote debugging using :1234
0x0000000000000000 in ?? ()
(gdb) b main
Breakpoint 1 at 0x802000f8: file dhrystone.c, line 184.
(gdb) c
Continuing.

Breakpoint 1, main (argc=0, argv=0x82200000) at dhrystone.c:184
184      REG   One_Fifty      Int_2_Loc=0;
(gdb) si
196      Next_Ptr_Glob = (Rec_Pointer) &x;
(gdb) info address main
Symbol "main" is a function at address 0x802000e6.
(gdb) i r
ra          0x80006b0c      0x80006b0c
sp          0x881ffff10     0x881ffff10
gp          0x80201800     0x80201800 <Arr_2_Glob+3136>
tp          0x8001de00     0x8001de00
t0          0x1            1
t1          0x82200000     2183135232
t2          0x82200000     2183135232
fp          0x88200000     0x88200000
s1          0x8000000000006800 -9223372036854749184
a0          0x0            0
a1          0x82200000     2183135232
a2          0x80200000     2149580800
a3          0x1            1
a4          0x0            0
a5          0x0            0
a6          0x80200000     2149580800
```

Figure 7.1: The interface of GDB

```
qemu-system-riscv64 -M virt -cpu rv64 -bios default -nographic -machine virt -kernel linux/arch/riscv/boot/Image -append "root=/dev/ada rw console=ttyS0" -drive file=rootsfs/oot.dta,format=rw,if=ahci -device virtio-blk-device,drive=hda0 -bios default

qemu-system-riscv64
OpenSBI
Platform Name      : QEMU VIRT Machine
Platform Name      : RISC-V
Platform Name      : S
Current Host       : 0
Firmware Base      : 0x00000000
Firmware Size      : 128 KB
Firmware SHA Version : 0.2
HID0: 0x0000000000000000
HID1: 0x0000000000000000
HID2: 0x0000000000000000
HID3: 0x0000000000000000
HID4: 0x0000000000000000
HID5: 0x0000000000000000
HID6: 0x0000000000000000
HID7: 0x0000000000000000
HID8: 0x0000000000000000
HID9: 0x0000000000000000
HID10: 0x0000000000000000
HID11: 0x0000000000000000
HID12: 0x0000000000000000
HID13: 0x0000000000000000
HID14: 0x0000000000000000
HID15: 0x0000000000000000
HID16: 0x0000000000000000
HID17: 0x0000000000000000
HID18: 0x0000000000000000
HID19: 0x0000000000000000
HID20: 0x0000000000000000
HID21: 0x0000000000000000
HID22: 0x0000000000000000
HID23: 0x0000000000000000
HID24: 0x0000000000000000
HID25: 0x0000000000000000
HID26: 0x0000000000000000
HID27: 0x0000000000000000
HID28: 0x0000000000000000
HID29: 0x0000000000000000
HID30: 0x0000000000000000
HID31: 0x0000000000000000
HID32: 0x0000000000000000
HID33: 0x0000000000000000
HID34: 0x0000000000000000
HID35: 0x0000000000000000
HID36: 0x0000000000000000
HID37: 0x0000000000000000
HID38: 0x0000000000000000
HID39: 0x0000000000000000
HID40: 0x0000000000000000
HID41: 0x0000000000000000
HID42: 0x0000000000000000
HID43: 0x0000000000000000
HID44: 0x0000000000000000
HID45: 0x0000000000000000
HID46: 0x0000000000000000
HID47: 0x0000000000000000
HID48: 0x0000000000000000
HID49: 0x0000000000000000
HID50: 0x0000000000000000
HID51: 0x0000000000000000
HID52: 0x0000000000000000
HID53: 0x0000000000000000
HID54: 0x0000000000000000
HID55: 0x0000000000000000
HID56: 0x0000000000000000
HID57: 0x0000000000000000
HID58: 0x0000000000000000
HID59: 0x0000000000000000
HID60: 0x0000000000000000
HID61: 0x0000000000000000
HID62: 0x0000000000000000
HID63: 0x0000000000000000
HID64: 0x0000000000000000
HID65: 0x0000000000000000
HID66: 0x0000000000000000
HID67: 0x0000000000000000
HID68: 0x0000000000000000
HID69: 0x0000000000000000
HID70: 0x0000000000000000
HID71: 0x0000000000000000
HID72: 0x0000000000000000
HID73: 0x0000000000000000
HID74: 0x0000000000000000
HID75: 0x0000000000000000
HID76: 0x0000000000000000
HID77: 0x0000000000000000
HID78: 0x0000000000000000
HID79: 0x0000000000000000
HID80: 0x0000000000000000
HID81: 0x0000000000000000
HID82: 0x0000000000000000
HID83: 0x0000000000000000
HID84: 0x0000000000000000
HID85: 0x0000000000000000
HID86: 0x0000000000000000
HID87: 0x0000000000000000
HID88: 0x0000000000000000
HID89: 0x0000000000000000
HID90: 0x0000000000000000
HID91: 0x0000000000000000
HID92: 0x0000000000000000
HID93: 0x0000000000000000
HID94: 0x0000000000000000
HID95: 0x0000000000000000
HID96: 0x0000000000000000
HID97: 0x0000000000000000
HID98: 0x0000000000000000
HID99: 0x0000000000000000
HID100: 0x0000000000000000
HID101: 0x0000000000000000
HID102: 0x0000000000000000
HID103: 0x0000000000000000
HID104: 0x0000000000000000
HID105: 0x0000000000000000
HID106: 0x0000000000000000
HID107: 0x0000000000000000
HID108: 0x0000000000000000
HID109: 0x0000000000000000
HID110: 0x0000000000000000
HID111: 0x0000000000000000
HID112: 0x0000000000000000
HID113: 0x0000000000000000
HID114: 0x0000000000000000
HID115: 0x0000000000000000
HID116: 0x0000000000000000
HID117: 0x0000000000000000
HID118: 0x0000000000000000
HID119: 0x0000000000000000
HID120: 0x0000000000000000
HID121: 0x0000000000000000
HID122: 0x0000000000000000
HID123: 0x0000000000000000
HID124: 0x0000000000000000
HID125: 0x0000000000000000
HID126: 0x0000000000000000
HID127: 0x0000000000000000
HID128: 0x0000000000000000
HID129: 0x0000000000000000
HID130: 0x0000000000000000
HID131: 0x0000000000000000
HID132: 0x0000000000000000
HID133: 0x0000000000000000
HID134: 0x0000000000000000
HID135: 0x0000000000000000
HID136: 0x0000000000000000
HID137: 0x0000000000000000
HID138: 0x0000000000000000
HID139: 0x0000000000000000
HID140: 0x0000000000000000
HID141: 0x0000000000000000
HID142: 0x0000000000000000
HID143: 0x0000000000000000
HID144: 0x0000000000000000
HID145: 0x0000000000000000
HID146: 0x0000000000000000
HID147: 0x0000000000000000
HID148: 0x0000000000000000
HID149: 0x0000000000000000
HID150: 0x0000000000000000
HID151: 0x0000000000000000
HID152: 0x0000000000000000
HID153: 0x0000000000000000
HID154: 0x0000000000000000
HID155: 0x0000000000000000
HID156: 0x0000000000000000
HID157: 0x0000000000000000
HID158: 0x0000000000000000
HID159: 0x0000000000000000
HID160: 0x0000000000000000
HID161: 0x0000000000000000
HID162: 0x0000000000000000
HID163: 0x0000000000000000
HID164: 0x0000000000000000
HID165: 0x0000000000000000
HID166: 0x0000000000000000
HID167: 0x0000000000000000
HID168: 0x0000000000000000
HID169: 0x0000000000000000
HID170: 0x0000000000000000
HID171: 0x0000000000000000
HID172: 0x0000000000000000
HID173: 0x0000000000000000
HID174: 0x0000000000000000
HID175: 0x0000000000000000
HID176: 0x0000000000000000
HID177: 0x0000000000000000
HID178: 0x0000000000000000
HID179: 0x0000000000000000
HID180: 0x0000000000000000
HID181: 0x0000000000000000
HID182: 0x0000000000000000
HID183: 0x0000000000000000
HID184: 0x0000000000000000
HID185: 0x0000000000000000
HID186: 0x0000000000000000
HID187: 0x0000000000000000
HID188: 0x0000000000000000
HID189: 0x0000000000000000
HID190: 0x0000000000000000
HID191: 0x0000000000000000
HID192: 0x0000000000000000
HID193: 0x0000000000000000
HID194: 0x0000000000000000
HID195: 0x0000000000000000
HID196: 0x0000000000000000
HID197: 0x0000000000000000
HID198: 0x0000000000000000
HID199: 0x0000000000000000
HID200: 0x0000000000000000
HID201: 0x0000000000000000
HID202: 0x0000000000000000
HID203: 0x0000000000000000
HID204: 0x0000000000000000
HID205: 0x0000000000000000
HID206: 0x0000000000000000
HID207: 0x0000000000000000
HID208: 0x0000000000000000
HID209: 0x0000000000000000
HID210: 0x0000000000000000
HID211: 0x0000000000000000
HID212: 0x0000000000000000
HID213: 0x0000000000000000
HID214: 0x0000000000000000
HID215: 0x0000000000000000
HID216: 0x0000000000000000
HID217: 0x0000000000000000
HID218: 0x0000000000000000
HID219: 0x0000000000000000
HID220: 0x0000000000000000
HID221: 0x0000000000000000
HID222: 0x0000000000000000
HID223: 0x0000000000000000
HID224: 0x0000000000000000
HID225: 0x0000000000000000
HID226: 0x0000000000000000
HID227: 0x0000000000000000
HID228: 0x0000000000000000
HID229: 0x0000000000000000
HID230: 0x0000000000000000
HID231: 0x0000000000000000
HID232: 0x0000000000000000
HID233: 0x0000000000000000
HID234: 0x0000000000000000
HID235: 0x0000000000000000
HID236: 0x0000000000000000
HID237: 0x0000000000000000
HID238: 0x0000000000000000
HID239: 0x0000000000000000
HID240: 0x0000000000000000
HID241: 0x0000000000000000
HID242: 0x0000000000000000
HID243: 0x0000000000000000
HID244: 0x0000000000000000
HID245: 0x0000000000000000
HID246: 0x0000000000000000
HID247: 0x0000000000000000
HID248: 0x0000000000000000
HID249: 0x0000000000000000
HID250: 0x0000000000000000
HID251: 0x0000000000000000
HID252: 0x0000000000000000
HID253: 0x0000000000000000
HID254: 0x0000000000000000
HID255: 0x0000000000000000
HID256: 0x0000000000000000
HID257: 0x0000000000000000
HID258: 0x0000000000000000
HID259: 0x0000000000000000
HID260: 0x0000000000000000
HID261: 0x0000000000000000
HID262: 0x0000000000000000
HID263: 0x0000000000000000
HID264: 0x0000000000000000
HID265: 0x0000000000000000
HID266: 0x0000000000000000
HID267: 0x0000000000000000
HID268: 0x0000000000000000
HID269: 0x0000000000000000
HID270: 0x0000000000000000
HID271: 0x0000000000000000
HID272: 0x0000000000000000
HID273: 0x0000000000000000
HID274: 0x0000000000000000
HID275: 0x0000000000000000
HID276: 0x0000000000000000
HID277: 0x0000000000000000
HID278: 0x0000000000000000
HID279: 0x0000000000000000
HID280: 0x0000000000000000
HID281: 0x0000000000000000
HID282: 0x0000000000000000
HID283: 0x0000000000000000
HID284: 0x0000000000000000
HID285: 0x0000000000000000
HID286: 0x0000000000000000
HID287: 0x0000000000000000
HID288: 0x0000000000000000
HID289: 0x0000000000000000
HID290: 0x0000000000000000
HID291: 0x0000000000000000
HID292: 0x0000000000000000
HID293: 0x0000000000000000
HID294: 0x0000000000000000
HID295: 0x0000000000000000
HID296: 0x0000000000000000
HID297: 0x0000000000000000
HID298: 0x0000000000000000
HID299: 0x0000000000000000
HID300: 0x0000000000000000
HID301: 0x0000000000000000
HID302: 0x0000000000000000
HID303: 0x0000000000000000
HID304: 0x0000000000000000
HID305: 0x0000000000000000
HID306: 0x0000000000000000
HID307: 0x0000000000000000
HID308: 0x0000000000000000
HID309: 0x0000000000000000
HID310: 0x0000000000000000
HID311: 0x0000000000000000
HID312: 0x0000000000000000
HID313: 0x0000000000000000
HID314: 0x0000000000000000
HID315: 0x0000000000000000
HID316: 0x0000000000000000
HID317: 0x0000000000000000
HID318: 0x0000000000000000
HID319: 0x0000000000000000
HID320: 0x0000000000000000
HID321: 0x0000000000000000
HID322: 0x0000000000000000
HID323: 0x0000000000000000
HID324: 0x0000000000000000
HID325: 0x0000000000000000
HID326: 0x0000000000000000
HID327: 0x0000000000000000
HID328: 0x0000000000000000
HID329: 0x0000000000000000
HID330: 0x0000000000000000
HID331: 0x0000000000000000
HID332: 0x0000000000000000
HID333: 0x0000000000000000
HID334: 0x0000000000000000
HID335: 0x0000000000000000
HID336: 0x0000000000000000
HID337: 0x0000000000000000
HID338: 0x0000000000000000
HID339: 0x0000000000000000
HID340: 0x0000000000000000
HID341: 0x0000000000000000
HID342: 0x0000000000000000
HID343: 0x0000000000000000
HID344: 0x0000000000000000
HID345: 0x0000000000000000
HID346: 0x0000000000000000
HID347: 0x0000000000000000
HID348: 0x0000000000000000
HID349: 0x0000000000000000
HID350: 0x0000000000000000
HID351: 0x0000000000000000
HID352: 0x0000000000000000
HID353: 0x0000000000000000
HID354: 0x0000000000000000
HID355: 0x0000000000000000
HID356: 0x0000000000000000
HID357: 0x0000000000000000
HID358: 0x0000000000000000
HID359: 0x0000000000000000
HID360: 0x0000000000000000
HID361: 0x0000000000000000
HID362: 0x0000000000000000
HID363: 0x0000000000000000
HID364: 0x0000000000000000
HID365: 0x0000000000000000
HID366: 0x0000000000000000
HID367: 0x0000000000000000
HID368: 0x0000000000000000
HID369: 0x0000000000000000
HID370: 0x0000000000000000
HID371: 0x0000000000000000
HID372: 0x0000000000000000
HID373: 0x0000000000000000
HID374: 0x0000000000000000
HID375: 0x0000000000000000
HID376: 0x0000000000000000
HID377: 0x0000000000000000
HID378: 0x0000000000000000
HID379: 0x0000000000000000
HID380: 0x0000000000000000
HID381: 0x0000000000000000
HID382: 0x0000000000000000
HID383: 0x0000000000000000
HID384: 0x0000000000000000
HID385: 0x0000000000000000
HID386: 0x0000000000000000
HID387: 0x0000000000000000
HID388: 0x0000000000000000
HID389: 0x0000000000000000
HID390: 0x0000000000000000
HID391: 0x0000000000000000
HID392: 0x0000000000000000
HID393: 0x0000000000000000
HID394: 0x0000000000000000
HID395: 0x0000000000000000
HID396: 0x0000000000000000
HID397: 0x0000000000000000
HID398: 0x0000000000000000
HID399: 0x0000000000000000
HID400: 0x0000000000000000
HID401: 0x0000000000000000
HID402: 0x0000000000000000
HID403: 0x0000000000000000
HID404: 0x0000000000000000
HID405: 0x0000000000000000
HID406: 0x0000000000000000
HID407: 0x0000000000000000
HID408: 0x0000000000000000
HID409: 0x0000000000000000
HID410: 0x0000000000000000
HID411: 0x0000000000000000
HID412: 0x0000000000000000
HID413: 0x0000000000000000
HID414: 0x0000000000000000
HID415: 0x0000000000000000
HID416: 0x0000000000000000
HID417: 0x0000000000000000
HID418: 0x0000000000000000
HID419: 0x0000000000000000
HID420: 0x0000000000000000
HID421: 0x0000000000000000
HID422: 0x0000000000000000
HID423: 0x0000000000000000
HID424: 0x0000000000000000
HID425: 0x0000000000000000
HID426: 0x0000000000000000
HID427: 0x0000000000000000
HID428: 0x0000000000000000
HID429: 0x0000000000000000
HID430: 0x0000000000000000
HID431: 0x0000000000000000
HID432: 0x0000000000000000
HID433: 0x0000000000000000
HID434: 0x0000000000000000
HID435: 0x0000000000000000
HID436: 0x00
```

Chapter 8

Results and discussions

This chapter analyzes the results of running the different benchmarks on QEMU. These benchmarks are compiled based on different RISC-V instruction sets. The compiled benchmarks are tested on QEMU with two different modes (user mode and system mode). Finally, we analyze the test results based on the characteristics of the benchmark.

8.1 Test environment

To make the test results relatively accurate, the following provisions are made for the test environment to minimize errors.

- QEMU based on two different processors runs on Ubuntu 18.04.
- The compiler used for the benchmarks are “*riscv64-unknown-elf*” for the QEMU that uses the 64-bit RISC-V processor and “*riscv32-unknown-elf*” for the QEMU that uses the 32-bit RISC-V processor. However, the benchmark “*Neural Networks*” and “*Imageproc*” must use functions such as “*fopen()*” or “*malloc()*” in the C program. This compiler does not support those functions. Hence, those two benchmarks use another compiler, “*riscv32-unknown-linux-gnu*” to generate executed binary file.
- When using the system mode to do the test, all benchmarks are put in the Linux system simulated by QEMU.
- All results were measured using the RV32GC or RV64GC platform.

- Because the QEMU does not have a function to get the number of running instructions, another simulator named Imperas can be used to obtain the counts of the instructions. But for the benchmarks “*nn*” and “*Imageproc*”, Imperas is incompatible with the compiler they are using. Therefore, those two benchmarks cannot get the counts of the instructions.
- Due to the different size of each benchmark, this results in a different number of instructions compiled by the compiler. To facilitate comparison, I use the unit Million Instructions Per Second (MIPS) to quantify the speed. The calculation method of MIPS in this paper is:

$$\text{MIPS} = \frac{\text{number of instructions}}{\text{time of System Mode(second)} \times 1000000} \quad (8.1)$$

8.2 Fibonacci sequence

Fib				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	52.105ss	70.53s	35,771,844,788	507
RV64IM	52.563s	69.87s	35,771,844,788	511.977
RV64IMFD	53.013s	72.05s	35,771,844,788	496.486
RV32IM	34.323s	52.49s	30,160,574,967	574.597
RV32IMFD	32.455s	55.19s	30,160,574,967	546.486

Table 8.1: The result of Fib benchmark

According to this table, it is easy to find that RV64I^{*}, RV64IM[†], and RV64IMFD[‡] have the same number of instructions. Similarly, the counts of the instructions of the RV32IM are the same as RV32IMFD. This is because the benchmark *Fibonacci sequence* only includes one integer data type and does not include multiplication and division operations. RV64IM/IMFD has more instructions than RV32IM/IMFD. This is because for the 64-bit platform, it needs to use extra instruction which can extend the data to 64-bit before completing the operation for the 32-bit data. Nevertheless, the 32-bit platform does not need this extra operation.

The run time of the RV64I is the almost same as RV64IM and RV64IMFD. Furthermore, the RV32IM is almost the same run time as RV32IMFD. This is

* I: integer † M: multiplication ‡ FD: float&double

because they have the same number of instructions. Furthermore, the run time of the user mode is less than that of the system mode. It is easy to understand that the user mode only tests the processor, but the system mode needs to consider the whole structure of the platform.

The speed of the 32-bit platform is the same, and the speed of the 64-bit platform is the same. This is because the benchmark *Fibonacci sequence* includes one integer data type and does not include multiplication and division operations and floating-point data. The output files of the compiled benchmark based on these three instruction sets (RV64I, RV64IM and RV64IMFD) are the same. The speed of the RV32IM/IMFD is faster than RV64IM/IMFD. It is easy to understand that RV32 only needs to deal with 32-bit data, but RV64 needs to process 64-bit data. Hence, when doing the same operation such as load, add, and store, RV32 must be faster than RV64.

8.3 Dhrystone

Dhrystone				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	58.989s	77.53s	34,650,003,031	446.924
RV64IM	50.060s	78.29s	34,650,003,031	442.585
RV64IMFD	49.945s	82.15s	34,650,003,031	421.789
RV32IM	31.804s	68.45s	32,450,004,233	474.069
RV32IMFD	30.404s	71.89s	32,450,004,233	451.384

Table 8.2: The result of Dhrystone

According to this table, it is easy to find that RV64I has more instructions than RV64IM and RV64IMFD. Furthermore, the counts of the instructions of the RV32IM are the same as RV32IMFD. It is because the benchmark *Dhrystone* only includes one integer data type, and it does not include multiplication and division operations. RV64IM/IMFD has more instruction than RV32IM/IMFD. The reason is the same as the section 8.2

The run time of the RV64I is the almost same as RV64IM and RV64IMFD. Furthermore, the RV32IM is almost the same run time as RV32IMFD. This is because they have the same number of instructions. Furthermore, the run time of the user mode is less than that of the system mode. The reason is the same as the section 8.2.

The speed of the 32-bit platform is the same, and the speed of the 64-bit platform is the same. This is because the benchmark *Dhrystone* includes one

integer data type and does not include multiplication and division operations and floating-point data. The output files of compiled benchmark based on these three instruction sets(RV64I, RV64IM and RV64IMFD) are the same. The speed of the RV32IM/IMFD is faster than RV64IM/IMFD. The reason has been mentioned in section 8.2.

8.4 Linpack

Linpack				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	37.005s	49.82s	55,647,183,188	1,116.965
RV64IM	25.897s	37.55s	30,305,176,393	807.062
RV64IMFD	10.369s	11.12s	4,487,083,201	403.515
RV32IM	15.353s	38.36s	32,162,270,942	838.433
RV32IMFD	8.104s	9.03s	3,922,262,858	434.359

Table 8.3: The result of Linpack

According to the table above, it is easy to find that RV64I has more instructions than RV64IM, and RV64IM has more instructions than RV64IMFD. The counts of the instructions of the RV32IM are more than RV32IMFD. The extension "I", "M", and "FD" provide optimization for integer, multiplication, and floating operations, respectively. As benchmark *Linpack* contains multiplication and floating point operations, ISAs that lack corresponding extensions spend more instructions to obtain an equivalent result. RV32IM has more instructions than RV64IM, but the RV32IMFD has fewer instructions than RV64IMFD. There are two factors that affect the number of instruction count. On the one hand, the 64-bit processor has 64-bit registers instead of 32-bit registers. When the processor execute the store/load instructions, the 64-bit processor can use the instruction "*ld*"(load double word) rather than "*lw*"(load word). This benchmark *Linpack* produces a very large matrix of floating point numbers. Therefore, the instruction counts can be reduced because of the instruction "*ld*"(load double word). On the other hand, the 64-bit platform still has some 32-bit data. As is mentioned in section 8.2, it needs to use extended instruction which can extend the data to 64-bit before completing the operation for the 32-bit data. Therefore, when the instruction counts is large, The first factor is the dominant factor and when the instruction counts are small, the second factor is the dominant factor for this benchmark.

Hence, RV32IM has more instructions than RV64IM, but RV32IMFD has fewer instructions than RV64IMFD.

The run time of the RV64I is longer than RV64IM, and the RV64IM is longer than RV64IMFD. Similarly, the RV32IM is longer than the RV32IMFD. This is because the instruction counts become fewer with the increase of ISA extensions. Furthermore, same as the section 8.2, the run time of the user mode is less than that of the system mode.

The speed of the RV64I is faster than RV64IM, and the RV64IM is faster than RV64IMFD. This is because RV64IM and RV64IMFD contain more complex instructions that take longer to run than instructions in RV64I. Similarly, the speed of RV32IM is faster than RV32IMFD. Furthermore, the speed of the RV32IM/IMFD is faster than RV64IM/IMFD. The reason has been mentioned in section 8.2.

8.5 Speed

Speed				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	57.511s	66.99s	55,620,000,327	830.273
RV64IM	14.209s	37.57s	35,370,000,325	941.443
RV64IMFD	14.534s	37.56s	35,370,000,325	941.693
RV32IM	6.096s	21.78s	32,130,000,315	1,475.207
RV32IMFD	6.027s	21.22s	32,130,000,315	1,514.138

Table 8.4: The result of Speed

According to this table, it is easy to find that RV64I has more instructions than RV64IM and RV64IMFD. Furthermore, the counts of the instructions of the RV32IM are the same as RV32IMFD. It is because the benchmark *Speed* only includes one integer data type and the multiplication and division operations and does not have a floating-point data type. Furthermore, RV64I has more instruction than RV32I, and RV64IM and RV64IMFD have more instruction than RV32IM and RV32IMFD relatively. The reason is the same as the section 8.2.

The run time of the RV64I is longer than RV64IM and RV64IMFD. It is because the instruction counts of RV64I is larger than the RV64IM and RV64IMFD. Furthermore, the RV32IM is almost the same run time as RV32IMFD. It is because the output files of compiled benchmarks based on

these two instruction sets(RV32IM and RV32IMFD) are the same. Moreover, same as the section 8.2, the run time of the user mode is less than that of the system mode.

About the speed of running instructions, the RV64I is lower than RV64IM and RV64IMFD. The RV32IM and RV32IMFD can run same number of instructions per unit time and the RV64IM and RV64IMFD can run same number of instructions per unit time. The reason why the RV32IM and RV32IMFD can run same number of instructions per unit time is that this benchmark does not have floating points data type. The output files of compiled benchmark based on these two instruction sets(RV64IM and RV64IMFD) are the same. For most benchmarks, RV64I can run same or more instructions than RV64IM in same time. But for the benchmark *Speed*, the RV64IM can run more instructions then RV64I in same time. It is because although the QEMU is a virtual platform, it still runs on the host machine with a high-performance processor. This high-performance processor can make the run time of the multiplication operation almost close to that of the addition operation. Hence, the RV64I is slower than RV64IM and RV64IMFD. Furthermore, the RV32 can run more instructions per unit time than RV64 when the same extension instruction set type is used. The reason has been mentioned in section 8.2.

8.6 Matrix multiplication

Matrix multiplication				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	63.342s	237.87s	73,685,110,868	309.771
RV64IM	40.212s	179.24s	27,205,174,781	151.781
RV64IMFD	41.75s	179.68s	27,205,174,781	151.409
RV32IM	3.951s	140.44s	24,647,075,546	175.499
RV32IMFD	4.185s	138.42s	24,647,075,546	178.060

Table 8.5: The result of Matrix multiplication

According to this table, it is easy to find that RV64I has more instructions than RV64IM and RV64IMFD. Furthermore, the counts of the instructions of the RV32IM are the same as RV32IMFD. It is because the benchmark *Matrix multiplication* only includes one integer data type and multiplication and division operations. Furthermore, same as the section 8.5, RV64I has more instruction than RV32I, and RV64IM and RV64IMFD have more instruction

than RV32IM and RV32IMFD relatively. The reason is the same as the section 8.2.

The regulation of run time for this benchmark is the same as the section 8.5. The run time of the RV64I is longer than RV64IM and RV64IMFD. Moreover, the RV32IM results in almost the same run time as RV32IMFD. Furthermore, same as the section 8.2, the run time of the user mode is less than that of the system mode.

The speed of the RV64I is faster than RV64IM and RV64IMFD. This is because RV64IM and RV64IMFD contain more complex instructions that take longer to run than instructions in RV64I. Furthermore, the speed of RV32IM and RV32IMFD is the same, and the speed of RV64IM and RV64IMFD is the same. Because this benchmark *Matrix Multiplication* does not have floating point data type. The output file of the compiled benchmark based on these two instruction sets(RV64IM and RV64IMFD) is the same. Moreover, the speed of the RV32IM/IMFD is faster than RV64IM/IMFD. The reason has been mentioned in section 8.2.

8.7 Signal filter

Signal filter				
ISA	User-mode	System-mode	Instructions	speed[MIPS]
RV64I	237.292s	258.44s	508,645,271,854	1,968.137
RV64IM	7.186s	17.07s	33,372,083,681	1,955.014
RV64IMFD	6.823s	17.51s	33,372,083,681	1,905.887
RV32IM	5.206s	13.69s	27,972,070,188	2,043.248
RV32IMFD	4.973s	13.86s	27,972,070,188	2,018.187

Table 8.6: The result of Signal filter

According to this table, it is easy to find that RV64I has more instructions than RV64IM and RV64IMFD. Moreover, the counts of the instructions of the RV32IM are the same as RV32IMFD. It is because the benchmark *Signal filter* only includes one integer data type and multiplication and division operations. Furthermore, same as the section 8.5, RV64I has more instruction than RV32I, and RV64IM and RV64IMFD have more instruction than RV32IM and RV32IMFD relatively. The reason is the same as the section 8.2.

The regulation of run time for this benchmark is the same as the section 8.5. The run time of the RV64I is longer than RV64IM and RV64IMFD.

Moreover, the RV32IM has almost the same run time the same as RV32IMFD. Furthermore, same as the section 8.2, the run time of the user mode is less than that of the system mode.

It has the same regulations about speed as the section 8.6. The speed of the RV64I is faster than RV64IM and RV64IMFD. The speed of RV32IM and RV32IMFD is the same, and the speed of RV64IM and RV64IMFD is the same. The speed of the RV32IM/IMFD is faster than RV64IM/IMFD. The reason has been mentioned in section 8.2.

8.8 Neural Networks

Neural Networks		
ISA	User-mode	System-mode
RV64I	14.034s	20.69s
RV64IM	13.368s	20.23s
RV64IMFD	8.613s	11.06s
RV32IM	11.255s	24.06s
RV32IMFD	7.099s	15.82s

Table 8.7: The result of Neural Net

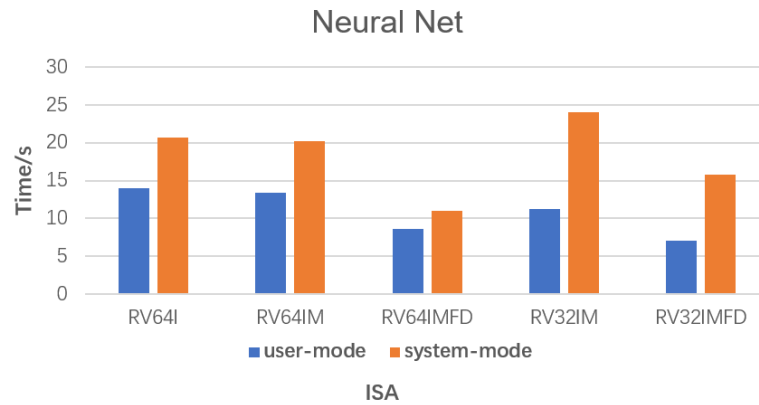


Figure 8.1: The result of Neural Networks

Due to the different compilers, this benchmark *Neural Networks* cannot obtain information about the number of instructions. According to the table 8.7 and the figure 8.1, the run time of the RV64I is slower than RV64IM, and

RV64IM is longer than RV64IMFD. Moreover, the RV32IM is slower than RV32IMFD in running time. It is because the benchmark *Neural Networks* includes the data types of integer and float, and it includes multiplication and division operations. The instructions must be reduced when the RISC-V instruction set changes from RV64I to RV64IMFD. And the run time of the user mode is less than that of the system mode. It is easy to understand that the user mode only tests the processor, but the system mode needs to consider the whole structure of the platform.

8.9 Imageproc

Due to the different compilers, this benchmark cannot obtain information about the number of instructions. According to this table 8.8 and figure 8.2, the run time of the RV64I is longer than RV64IM and RV64IMFD. Furthermore, the RV32IM is the same as RV32IMFD in run time. It is because the benchmark *Imageproc* only includes the integer data type and multiplication and division operations. The instructions must reduce when the RISC-V instruction set change from RV64I to RV64IM, and the number of instructions of RV64IM is the same as RV64IMFD. And the run time of the user mode is less than that of the system mode. The reason is the same as the section 8.2.

Imageproc		
ISA	User-mode	System-mode
RV64I	50.232s	81.23s
RV64IM	18.602s	31.78s
RV64IMFD	17.917s	31.97s
RV32IM	13.878s	33.69s
RV32IMFD	14.174s	33.33s

Table 8.8: The result of Image processing

8.10 FFT

According to the table 8.9, it is easy to find that RV64I has more instructions than RV64IM, and RV64IM has more instructions than RV64IMFD. The counts of the instructions of the RV32IM are more than RV32IMFD. It is because the benchmark *FFT* includes the data type of integer and float

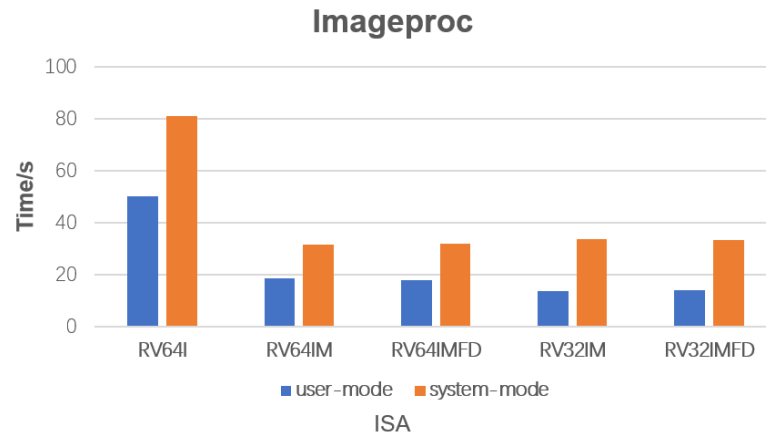


Figure 8.2: The result of Image processing

FFT				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	58.518s	79.72s	80,887,682,164	1,014.647
RV64IM	18.254s	32.39s	20,593,458,775	635.797
RV64IMFD	8.851s	20.59s	2,302,951,947	111.848
RV32IM	18.610s	43.09s	30,989,349,235	719.177
RV32IMFD	14.617s	21.26s	2,472,754,073	116.310

Table 8.9: The result of FFT

and multiplication and division operations. Furthermore, RV32IM has more instructions than RV64IM, and the RV32IMFD has more instructions than RV64IMFD. As is mentioned in the section 8.4, the 64-bit processor can use the instruction “*ld*”(load double word) rather than “*lw*”(load word) to reduce the instruction counts. Unlike other benchmarks, this benchmark has many header files which consist of many data. And when running this benchmark, there are many load and store instructions needed. Therefore RV32IM has more instructions than RV64IM, and the RV32IMFD has more instructions than RV64IMFD.

The regulation of run time for this benchmark is the same as the section 8.4. The run time of the RV64I is longer than RV64IM, and the RV64IM is longer than RV64IMFD. Furthermore, the RV32IM is slower than the RV32IMFD. And the run time of the user mode is less than that of the system mode. The reason is the same as the section 8.2.

The regulation of speed for this benchmark is the same as the section 8.4

as well. The speed of the RV64I is faster than RV64IM, and the speed of the RV64IM is faster than that of RV64IMFD. The speed of RV32IM is faster than RV32IMFD. The speed of the RV32IM/IMFD is faster than RV64IM/IMFD. The reason is the same as the section 8.2.

8.11 Compression

Compression				
ISA	User-mode	System-mode	Instructions	Speed[MIPS]
RV64I	46.302s	77.51s	25,744,533,936	332.145
RV64IM	46.891s	73.56s	25,744,533,936	349.980
RV64IMFD	47.603s	73.11s	25,744,533,936	352.134
RV32IM	27.889s	68.28s	30,043,391,785	440.003
RV32IMFD	27.556s	69.03s	30,043,391,785	435.222

Table 8.10: The result of Compression

According to the table 8.10, it is easy to find that RV64I, RV64IM, and RV64IMFD have the same number of instructions. Similarly, the counts of the instructions of the RV32IM are the same as RV32IMFD. It is because the benchmark **Compression** only includes one integer data type and does not include multiplication and division operations. Same as the section 8.10, RV32IM has more instructions than RV64IM, and the RV32IMFD has more instructions than RV64IMFD. It is because this benchmark needs to load massive data from the head file so that the 64-bit processor can use the instruction “*ld*”(load double word) rather than “*lw*”(load word) to reduce the instruction counts.

The regulation of the run time of this benchmark is the same as the section 8.2. The run time of the RV64I is the same as RV64IM and RV64IMFD. Furthermore, the RV32IM is almost the same as RV32IMFD in run time. And the run time of the user mode is less than that of the system mode. The reason is the same as the section 8.2.

The regulation of the speed of this benchmark is the same as the section 8.2 as well. The speed of the 32-bit platform is the same, and the speed of the 64-bit platform is the same. The speed of the RV32IM/IMFD is faster than RV64IM/IMFD.

8.12 Conclusion

After analyzing the data in the above section, there are some conclusions:

1. If the data types and operation types contained in the benchmark are relatively simple, e.g. integer only, the test results of the benchmark are consistent whatever instruction set the benchmark was compiled against.
2. If the data types and operation types contained in the benchmark are relatively complex, the test results of the benchmark will become better and better with the expansion of the instruction set more and more comprehensive.
3. When the benchmark includes massive data needed to deal with, the 64-bit platform can use fewer instruction counts to run this benchmark on QEMU.
4. The user mode of the QEMU always needs less time to run the benchmark compared to the system mode. As described in chapter 2.1.2, the user mode only tests the processor, but the system mode needs to consider the whole architecture of the platform. When not considering the specific computer architecture and focusing only on the processor's needs, people use user mode to prioritize the function.

Chapter 9

Conclusion and Future Works

This thesis has completed a preliminary introduction to QEMU based on RISC-V processors. It includes how to install QEMU based on a RISC-V processor and describes in detail several important steps involved in the installation process. In addition, each step of the installation process is described in detail. In addition, this article also introduces the source code of the platform used in QEMU, including the memory space of distribution, and the loading of kernel and firmware.

Furthermore, ten different benchmarks are designed to test the performance of RISC-V processors in QEMU. These benchmarks involve different data types and algorithms. These benchmarks are then compiled with different RISC-V instruction sets and run on QEMU. Then the results of running benchmarks are analysed. After that, the thesis also conducts a bare metal test of QEMU based on a RISC-V processor, which involves how to change the starting address of the compiled file during the compilation process and uses GDB to test the benchmark run or not.

But some parts still need to improve in future work.

Firstly, the “*virt*” platform is a virtual platform used in this project. It has been introduced in the chapter 6. But this part only introduces a part of the source code about this platform and the source code of this platform does not include a cache design. If the user wants to study more about this platform, the source code of other parts of it needs to be understood. In addition, trying to build a new platform which can be run on the QEMU is necessary.

Secondly, on the aspect of multi-processing, QEMU supports sets of many cores. It is still an open question to explore how these cores can be multi-threaded.

Thirdly, this thesis has done the bare metal test on the QEMU but has not

found a suitable way to test the run time of the benchmarks on a Linux system. How to get the run time of the benchmark on QEMU without an operating system is still a challenge.

Fourth, for the benchmark neural network, a relatively small model with only three input nodes is used in this work. It will be necessary to build a more complex model with more nodes in the future.

In conclusion, the QEMU-RISCV structure is still underdeveloped, and it is also gradually improving. This thesis has conducted tests to some parts of this field, but there still exist more functions that need to be studied.

References

- [1] I. Pogarcic, D. Krnjak, and D. Ozanic, “Business benefits from the virtualization of an ict infrastructure,” *International Journal of Engineering Business Management*, vol. 4, no. 3, p. 42, 2012. [Page 1.]
- [2] G. O. Osman, “Emulating the internet of things with qemu.” [Online]. Available: <https://gupea.ub.gu.se/handle/2077/66920> [Page 1.]
- [3] F. Bellard, “Qemu, a fast and portable dynamic translator.” 01 2005, pp. 41–46. [Pages 1 and 8.]
- [4] B. BAILEY, “Why risc-v is succeeding? interest in this particular isa is expanding, but the growth of other open-source hardware is less certain,” 2021. [Online]. Available: <https://semiengineering.com/why-risc-v-is-succeeding> [Page 2.]
- [5] The QEMU Project Developers, “Qemu official document,documentation/platforms/riscv.” [Online]. Available: <https://wiki.qemu.org/Documentation/Platforms/RISCV> [Page 3.]
- [6] Qemu registered trademark of Fabrice Bellard, 2020. [Online]. Available: http://wiki.qemu.org/Main_Page [Page 5.]
- [7] “Official suse linux enterprise documentation virtualization guide chapter 26 qemu overview.” [Online]. Available: <https://documentation.suse.com/sles/12-SP4/html/SLES-all/cha-qemu-overview.html> [Page 5.]
- [8] Fuentes Morales, Jose Luis Bismarck, “Evaluating gem5 and qemu virtual platforms for arm multicore architectures,” Master’s thesis, KTH, School of Information and Communication Technology (ICT), 2016. [Pages 5, 7, and 8.]

- [9] Lovric, Dalibor and Olsson, Christian, “Virtual Controllers,” 2016, Student Paper. [Pages 5 and 8.]
- [10] H. Holm and M. Persson, “Qemu as a platform for plc virtualization an analysis from a cyber security perspective,” 2018. [Pages vii, 6, and 7.]
- [11] The QEMU Project Developers, “Qemu documentation release 7.0.50,” p. 289, 2022. [Online]. Available: <https://readthedocs.org/projects/qemu/downloads/pdf/latest/> [Pages 7 and 8.]
- [12] —, “Qemu documentation release 7.0.50,” p. 227, 2022. [Online]. Available: <https://readthedocs.org/projects/qemu/downloads/pdf/latest/> [Page 8.]
- [13] D. Patterson, “An evolving benchmark suite for embedded iot computers from an academic-industrial cooperative,” 2019. [Page 8.]
- [14] RISC-V Foundation, “Running 64- and 32-bit risc-v linux on qemu,” 2019-2020. [Page 10.]
- [15] “riscv-gnu-toolchain.” [Online]. Available: <https://github.com/riscv-collab/riscv-gnu-toolchain> [Pages 10 and 12.]
- [16] S. Zamani, “Investigation of using a pret processor on a low-cost, low-power fpga,” 2016. [Page 12.]
- [17] “Newlib.” [Online]. Available: <https://sourceware.org/newlib/> [Page 12.]
- [18] “Glibc.” [Online]. Available: <https://www.gnu.org/savannah-checkouts/gnu/libc/libc.html> [Page 12.]
- [19] C. Walls, “Chapter 9 - open source, embedded linux, and android,” in *Embedded Software (Second Edition)*, second edition ed., C. Walls, Ed. Oxford: Newnes, 2012, p. 339. ISBN 978-0-12-415822-1. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B978012415822100009X> [Page 15.]
- [20] N. Wells, “Busybox: A swiss army knife for linux,” *Linux Journal*, vol. 2000, p. 10, 01 2000. [Page 16.]
- [21] Busybox official website. [Online]. Available: <https://busybox.net/~landley/forensics.txt> [Page 16.]

- [22] A. Grigas, “The fibonacci sequence: Its history, significance, and manifestations in nature,” 2013. [Page 23.]
- [23] Richard York, “Tbenchmarking in context: Dhrystone,” 2002. [Online]. Available: <https://ocw.snu.ac.kr/sites/default/files/MATERIAL/2486.pdf> [Page 24.]
- [24] A. R. Weiss, “Dhrystone benchmark: History, analysis, scores and recommendations,” 2002. [Page 24.]
- [25] J. J. Dongarra, P. Luszczek, and A. Petitet, “The linpack benchmark: past, present and future,” *Concurrency and Computation: practice and experience*, vol. 15, no. 9, pp. 803–820, 2003. [Page 26.]
- [26] J. J. Dongarra, “The linpack benchmark: An explanation,” in *International Conference on Supercomputing*. Springer, 1987, pp. 456–474. [Page 26.]
- [27] L. Aksoy, C. Lazzari, E. Costa, P. Flores, and J. Monteiro, “Design of digit-serial fir filters: Algorithms, architectures, and a cad tool,” *Very Large Scale Integration (VLSI) Systems, IEEE Transactions on*, vol. 21, pp. 498–511, 03 2013. doi: 10.1109/TVLSI.2012.2188917 [Pages vii and 29.]
- [28] K. Gurney, *An introduction to neural networks*. CRC press, 2018. [Page 30.]
- [29] G. V. Mendonça, “Eduardo ab da silva,” *THE ELECTRICAL ENGINEERING HANDBOOK*, p. 891, 2004. [Page 31.]
- [30] Simplilearn, “What is image processing : Overview, applications, benefits, and who should learn it,” 2022. [Online]. Available: <https://www.simplilearn.com/image-processing-article> [Page 31.]
- [31] C. Van Loan, *Computational frameworks for the fast Fourier transform*. SIAM, 1992. [Page 34.]
- [32] The QEMU Project Developers, “Qemu documentation release 7.0.50,” p. 184, 2022. [Online]. Available: <https://readthedocs.org/projects/qemu/downloads/pdf/latest/> [Page 41.]

Appendix 1

```

uint32_t reset_vec[10] = {
    0x00000297,      /* 1: auipc t0, %pcrel_hi(fw_dyn) */
    0x02828613,      /* addi a2, t0, %pcrel_lo(1b) */
    0xf1402573,      /* csrr a0, mhartid */
#ifdef TARGET_RISCV32
    0x0202a583,      /* lw a1, 32(t0) */
    0x0182a283,      /* lw t0, 24(t0) */
#elif defined(TARGET_RISCV64)
    0x0202b583,      /* ld a1, 32(t0) */
    0x0182b283,      /* ld t0, 24(t0) */
#endif
    0x00028067,      /* jr t0 */
    start_addr,      /* start: .dword */
    start_addr_hi32,
    fdt_load_addr,   /* fdt_laddr: .dword */
    0x00000000,
                                /* fw_dyn: */

};

```

Appendix 2

```

.section .init, "ax"
.global _start
_start:
    .cfi_startproc
    .cfi_undefined ra
    .option push
    .option norelax
    la gp, __global_pointer$
    .option pop
    la sp, __stack_top
    add s0, sp, zero
    jal zero, main
    .cfi_endproc
.end

```

